

# Sleep-time Compute: 테스트 시점 추론 스케일링을 넘어서

Sleep-time Compute: Beyond Inference Scaling at Test-time

Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, Joseph E. Gonzalez

**문서 성격** 원문 순서 · 수치 · 한정어를 보존한 한국어 구조 보존형 의역본이다. 본문 뒤의 원문 보존 부록은 원 논문의 equation, table, figure, caption 을 검증 가능하게 남긴다. 인용과 재현에는 반드시 고정 원문을 사용한다.

**고정 버전** arXiv:2504.13171v1

**배포 상태** internal-only — arXiv license; translated derivative and complete source plate are not cleared for public redistribution

# Contents

|                                                      |          |
|------------------------------------------------------|----------|
| 초록 (Abstract) . . . . .                              | 2        |
| 1. 서론 (Introduction) . . . . .                       | 2        |
| 2. 관련 연구 (Related Work) . . . . .                    | 2        |
| 3. Sleep-time Compute . . . . .                      | 2        |
| 4. 실험 설정 (Experimental Setup) . . . . .              | 3        |
| 4.1 데이터셋 . . . . .                                   | 3        |
| 4.2 모델과 기준선 . . . . .                                | 3        |
| 5. 실험 결과 (Experiments and Results) . . . . .         | 3        |
| 5.1 정확도- test-time compute Pareto frontier . . . . . | 3        |
| 5.2 Sleep-time compute 의 스케일링 . . . . .              | 4        |
| 5.3 여러 질의에 걸친 상각 . . . . .                           | 4        |
| 5.4 질의 예측 가능성 . . . . .                              | 4        |
| 6. Agentic SWE 사례 (SWE-Features) . . . . .           | 4        |
| 7. 논의와 한계 (Discussion and Limitations) . . . . .     | 4        |
| 부록의 구성 . . . . .                                     | 5        |
| <b>번역 · 검증 고지</b>                                    | <b>6</b> |
| <b>원문 구조 · 그림 보존 부록</b>                              | <b>7</b> |

## 초록 (Abstract)

테스트 시점 계산 (test-time compute) 의 확장은 대규모 언어 모델이 어려운문제를 풀게 하는 핵심 수단으로 부상했지만, 높은 지연시간과 추론 비용을 수반한다. 이 논문은 질의가 제시되기 전에 모델이 문맥에 관해 오프라인으로 “생각” 하게 하는 **sleep-time compute** 를 제안한다. 앞으로 사용자가 물을 법한 질의를 예상하고 유용한 중간 산출물을 미리 계산하면, 실제 질의시 필요한 계산을 크게 줄일 수 있다는 생각이다.

저자들은 두 추론 과제를 상태 보존형으로 바꾼 Stateful GSM-Symbolic 과 Stateful AIME 을 만든다. 같은 정확도를 얻는 데 필요한 test-time compute 가 두 과제에서 약 **5 배** 줄었다. sleep-time compute 의 양을 늘리면 Stateful GSM-Symbolic 정확도는 최대 **13%**, Stateful AIME 정확도는 최대 **18%** 더 높아졌다. 하나의 문맥에 여러 관련 질의를 붙인 Multi-Query GSM-Symbolic 에서는 오프라인 계산을 질의들 사이에 상각해 평균 질의 비용을 **2.5 배** 낮췄다. 효과의 크기는 미래 질의가 얼마나 예측 가능한지와 잘 연관되었다. 마지막으로 실제적인 agentic software-engineering 과제에 적용하는 사례를 제시한다.

## 1. 서론 (Introduction)

추론 모델은 답을 내기 전에 더 많은 token 과 검색 분기를 사용함으로써 성능을 높인다. 그러나 사용자가 질의를 낸 뒤에 이 계산을 모두 수행하면 응답 지연과 비용이 직접 증가한다. 반대로 사용자나 에이전트의 문맥은 질의보다 먼저 알려지는 경우가 많다. 저장소, 문서 집합, 장기간 대화이력처럼 비교적 안정된 문맥은 먼저 도착하고, 그 문맥에 관한 구체적인 질문은 나중에 도착한다.

논문은 이 시간적 비대칭을 이용한다. 원시 문맥을 그대로 저장해 두었다가 매번 질의와 함께 처리하는 대신, 질의가 없는 시간에 모델이 문맥을 다시 읽고 계산·요약·추론 결과를 **학습된 문맥 (learned context)** 으로 기록한다. 이후 질의가 오면 원시 문맥과 학습된 문맥을 함께 사용한다. 여기서 “learned” 는 모델 가중치 갱신을 뜻하지 않는다. 이 논문의 기본 방법은 텍스트 형태의 외부 기억을 미리 만드는 추론 절차다.

핵심 질문은 세 가지다. 첫째, 오프라인 계산이 정확도-온라인 계산 Pareto frontier 를 실제로 이동시키는가. 둘째, sleep-time budget 을 늘리면 그 frontier 가 계속 개선되는가. 셋째, 하나의 문맥을 여러 질의가 공유할 때 미리 계산한 비용을 충분히 상각할 수 있는가.

## 2. 관련 연구 (Related Work)

논문은 test-time scaling, self-consistency, search 와 verifier 를 이용한 추론, 문맥 압축, retrieval 과 장기 에이전트 기억을 연결한다. 기존 test-time scaling 은 질의 이후에 계산을 늘린다. caching 은 동일 계산의 재사용을 겨냥하지만, 미래 질의에 도움이 될 새로운 추론 결과를 능동적으로 만들지는 않는다. 요약이나 retrieval 은 문맥 길이를 줄이거나 관련 조각을 찾지만, 아직 묻지 않은 질의에 대해 다단계 계산을 수행하는 것과 구별된다.

sleep-time compute 는 이들 방법을 배제하지 않는다. 원시 문맥의 검색 결과를 입력으로 받아 오프라인 추론을 할 수도 있고, 생성된 learned context 를 retrieval index 에 저장할 수도 있다. 논문이 분리해 측정하는 것은 “질의가 오기 전” 에 사용한 계산량이다.

## 3. Sleep-time Compute

원시 문맥을  $c$ , 나중에 도착하는 질의를  $q$ , sleep-time 계산 예산을  $B_s$ , test-time 계산 예산을  $B_t$  라고 하자. sleep 단계는 아직  $q$  를 보지 않은 채  $c$  를 처리하여 learned context 인  $\hat{c}$  를 만든다.

$$\hat{c} = S(c; B_s), \quad \hat{y} = A(c, \hat{c}, q; B_t).$$

목표는 ( $B_s$ )를 늘렸을 때 같은 정확도에 필요한 ( $B_t$ )가 줄거나, 같은 ( $B_t$ )에서 정확도가 높아지는지를 보는 것이다. 계산을 단순히 앞당긴것만으로는 총비용이 절감되지 않을 수 있으므로, 논문은 한 문맥에 대한질의 수 ( $N_q$ )도 명시적으로 고려한다. 질의당 평균 비용은 대략 다음과같이 상각된다.

$$C_{\text{avg}} = C_{\text{test}} + \frac{C_{\text{sleep}}}{N_q}.$$

오프라인 모델은 가능한 질문을 직접 나열하는 데 그치지 않고, 문맥 속수량 관계를 계산하고, 파생 사실을 만들고, 후속 답변에 필요한 중간 결과를기록한다. 예컨대 “800 개 공 중 1/4 이 테니스공, 그 절반이남색, 그중 1/10 에 표식” 이라는 문맥이라면 각 단계의 수량과최종 수량을 미리 계산한다. 나중에 어느 중간 단계가 질문되어도 짧은온라인 응답으로 답할 수 있다.

## 4. 실험 설정 (Experimental Setup)

### 4.1 데이터셋

**Stateful GSM-Symbolic** 은 수학 문장제의 배경 문맥과 구체 질의를분리한다. 모델은 sleep 단계에서 배경만 받고, test 단계에서 질문을받는다. **Stateful AIME** 도 어려운 수학 문제에서 질의 전에 사용할수 있는 상태 문맥을 구성한다. **Multi-Query GSM-Symbolic** 은 같은문맥에서 파생되는 여러 질문을 묶어, 한 번 만든 learned context 를 몇 번재사용할 수 있는지 측정한다.

SWE 사례에서는 software repository 와 pull-request 설명처럼 큰 상태를먼저 제공하고, 모델이 코드 구조와 잠재적으로 중요한 feature 를 미리분석한 뒤 실제 작업을 수행하게 한다. 이 평가는 수학 benchmark 의 통제된조건에서 벗어나 agent lifecycle 에 적용 가능성을 살펴보기 위한 사례연구다.

### 4.2 모델과기준선

GSM-Symbolic 에는 GPT-4o-mini 와 GPT-4o 를 사용한다. AIME 에는 o1, o3-mini, Claude Sonnet 3.7 Extended Thinking, DeepSeek-R1 처럼 test-time compute 를늘렸을 때 성능이 개선되는 모델을 포함한다. 주 기준선은 (c) 와 (q) 를 test time 에 처음 함께 보는 표준 설정이다. 문맥만 보고 정답을 추측할 수있는지를 확인하기 위해 context-only 기준선도 둔다.

sleep-time budget 과 test-time budget 을 각각 변화시키고 accuracy 와 token/compute 비용을 함께 표시한다. 따라서 어떤 조건이 단지총 계산량을 더 쓴 것인지, 온라인 지연을 줄인 것인지, 질의당 총비용까지낮춘 것인지 구분할 수 있다.

## 5. 실험 결과 (Experiments and Results)

### 5.1 정확도– test-time compute Pareto frontier

learned context 를 제공하면 같은 정확도에 도달하는 데 필요한 test-time compute 가 Stateful GSM-Symbolic 과 Stateful AIME 에서 약 5 분의 1 로 줄었다. 중요한 비교는 sleep 계산을 더한 시스템이 단순 기준선보다 총 token 을 많이썼는지가 아니라, 사용자가 기다리는 구간의 계산량과 정확도 사이 frontier 가 이동했는가이다. 결과는 여러 모델에서 그 이동을 보인다.

## 5.2 Sleep-time compute 의스케일링

sleep 단계에 더 큰 budget 을 주면 learned context 가 더 많은 파생 관계와 중간 해답을 포함한다. Stateful GSM-Symbolic 에서는 최대 13%, Stateful AIME 에서는 최대 18%의 정확도 상승이 보고된다. 다만 모든 문맥에서 단조롭게 같은 이득이 나오는 것은 아니다. 미래 질문과 무관한 계산을 많이 만들면 비용만 늘 수 있다.

## 5.3 여러 질의에 걸친상각

Multi-Query GSM-Symbolic 에서 동일 learned context 를 관련 질문들이 공유하면 sleep 비용이 분산된다. 보고된 설정에서 평균 질의 비용은 2.5 배 감소한다. 한 문맥당 질의가 하나뿐이면 오프라인 비용을 회수하기 어렵고, 반복적으로 조회되는 사용자 · 문서 · repository 상태일수록 경제성이 커진다는 뜻이다.

## 5.4 질의 예측가능성

저자들은 어떤 문맥에서 sleep-time compute 가 잘 작동하는지를 분석한다. 미래 질의의 분포가 문맥으로부터 어느 정도 예측될 때, 미리 계산한 항목이 실제 질문과 겹칠 확률이 높다. 반대로 가능한 질문 공간이 넓거나 질의가 문맥과 약하게 연결되면 learned context 가 적중하지 않는다. 논문은 질의 예측 가능성과 sleep-time 효과 사이의 상관을 보고하며, 이 변수를 방법의 적용 조건으로 제시한다.

## 6. Agentic SWE 사례 (SWE-Features)

software-engineering 에이전트는 문제를 받은 뒤 repository 를 탐색하고, 관련 파일을 읽고, 변경 계획을 세우며, 테스트 실패에 따라 다시 탐색한다. 논문은 repository 상태를 미리 분석하여 구현에 유용할 수 있는 feature 와 메모를 만들고, 실제 문제 해결 시 이를 사용하는 절차를 시험한다. 오프라인 산출물은 “rethink memory” 에 저장되고 이후 단계에서 검색된다.

이 사례는 완전한 자동 사전 해결이 아니라, 반복되는 codebase 이해 비용을 질의 이전으로 옮기는 시도다. 수학 과제보다 문맥과 질문의 불확실성이 크기 때문에 결과는 proof-of-concept 로 제시되며, 일반적인 SWE 성능 향상을 확정하는 실험으로 해석되지 않는다.

## 7. 논의와 한계 (Discussion and Limitations)

sleep-time compute 가 가능하려면 문맥이 질의보다 먼저 존재해야 하고, 그 문맥이 sleep 이후에도 충분히 안정적이어야 한다. 문맥이 바뀌면 learned context 를 무효화하거나 갱신해야 한다. 미래 질의가 예측 불가능하면 유용한 계산을 선별하기 어렵다. 오프라인 생성물이 잘못된 추론을 포함하면 이후 답변이 그 오류를 재사용할 수 있으므로 provenance 와 재검증이 필요하다.

온라인 지연 감소와 총비용 감소는 서로 다른 주장이다. 한 문맥에 적은 수의 질의만 오면 sleep 비용을 상각하지 못한다. 반대로 반복 질의가 많고 문맥 reuse 가 높으면 미리 계산한 비용이 여러 요청에 분산된다. 따라서 적절한 sleep budget 은 예상 질의 수, 질의 분포, 문맥 변화율, learned context 의 크기를 함께 고려해야 한다.

이 논문의 sleep 은 가중치 update, continual learning, 생물학적 NREM/REM 모사를 포함하지 않는다. 모델 외부 텍스트 기억을 오프라인으로 합성하는 inference-time 기법이다. 저자들이 입증한 범위는 통제된 reasoning benchmark 와 제한된 SWE 사례이며, 장기적인 개인화 · 망각 · 삭제 · 보안 · 모델용량 문제는 후속 연구 대상으로 남는다.

## 부록의 구성

원문 부록에는 각 benchmark prompt, Stateful AIME 예시, Multi-Query GSM-Symbolic 구성, SWE-Features 세부 절차와 prompt, 예측 가능한 질문과 예측하기 어려운 질문의 예, rethink memory 구현, 연도별 AIME 결과가 포함된다. 뒤의 원문 보존 부록은 이 모든 appendix 와 26 개 figure 및 표를 고정 v1 PDF 그대로 제공한다.

## 번역·검증 고지

이 문서는 분석이나 평가를 추가하지 않는 것을 원칙으로 하며, 원문의 주장 강도와 수치를 유지하도록 작성되었다. 한국어 초안 작성과 구조 검수에는 Codex GPT-5.6 을 사용했고, 고정 source hash·원문 보존 부록·역대조 표를 통해 검증했다. 불일치가 있을 때에는 원문이 우선한다.

## 원문 구조·그림 보존 부록

아래 부록은 고정 원문의 equation, table, figure, caption 과 section 배치를 대조하기 위한 검증판이다. arXiv license; translated derivative and complete source plate are not cleared for public redistribution

# Sleep-time Compute: Beyond Inference Scaling at Test-time

Kevin Lin<sup>1\*</sup> Charlie Snell<sup>2\*</sup>

Yu Wang<sup>1</sup> Charles Packer<sup>1</sup> Sarah Wooders<sup>1</sup> Ion Stoica<sup>1 2</sup> Joseph E. Gonzalez<sup>1 2</sup>

<sup>1</sup>Letta <sup>2</sup>University of California, Berkeley

research@letta.com

## Abstract

Scaling test-time compute has emerged as a key ingredient for enabling large language models (LLMs) to solve difficult problems, but comes with high latency and inference cost. We introduce sleep-time compute, which allows models to “think” offline about contexts before queries are presented: by anticipating what queries users might ask and pre-computing useful quantities, we can significantly reduce the compute requirements at test-time. To demonstrate the efficacy of our method, we create modified versions of two reasoning tasks – Stateful GSM-Symbolic and Stateful AIME. We find that sleep-time compute can reduce the amount of test-time compute needed to achieve the same accuracy by  $\sim 5\times$  on Stateful GSM-Symbolic and Stateful AIME and that by scaling sleep-time compute we can further increase accuracy by up to 13% on Stateful GSM-Symbolic and 18% on Stateful AIME. Furthermore, we introduce Multi-Query GSM-Symbolic, which extends GSM-Symbolic by including multiple related queries per context. By amortizing sleep-time compute across related queries about the same context using Multi-Query GSM-Symbolic, we can decrease the average cost per query by  $2.5\times$ . We then conduct additional analysis to understand when sleep-time compute is most effective, finding the predictability of the user query to be well correlated with the efficacy of sleep-time compute. Finally, we conduct a case-study of applying sleep-time compute to a realistic agentic SWE task. Code and data released at: <https://github.com/letta-ai/sleep-time-compute>.

## 1 Introduction

Test-time scaling has emerged as an effective way to boost LLM performance on challenging tasks by spending more time thinking on difficult problems (OpenAI, 2024; DeepSeek-AI, 2024; Snell et al., 2024; Brown et al., 2024). However, improved performance from test-time compute comes at a significant increase in latency and cost, waiting potentially several minutes for answers and costing up to tens of dollars per query.<sup>1</sup> These drawbacks are in part due to the fact that the current approach to applying test-time compute assumes that problems are stateless, i.e. queries (user queries at test-time) and the contexts (background information) required for answering them are provided to the model together at “test-time.” In practice, this means that if multiple related queries require making similar inferences about the context at “test-time,” the model will have to recompute redundant computations each time, incurring additional latency and cost.

In reality, many LLM applications are *inherently stateful*, and work in conjunction with persisted, re-used context. A classic example is document question-answering, where documents contextualize responses to questions. Coding agents also operate on a large common repository and participate in multiple rounds of debugging support, while conversational assistants need to maintain the past dialogue. In all these applications, there is context (available documents, a codebase, or conversation history) that is already available before the next user input.

<sup>1</sup><https://platform.openai.com/docs/models/o1-pro>

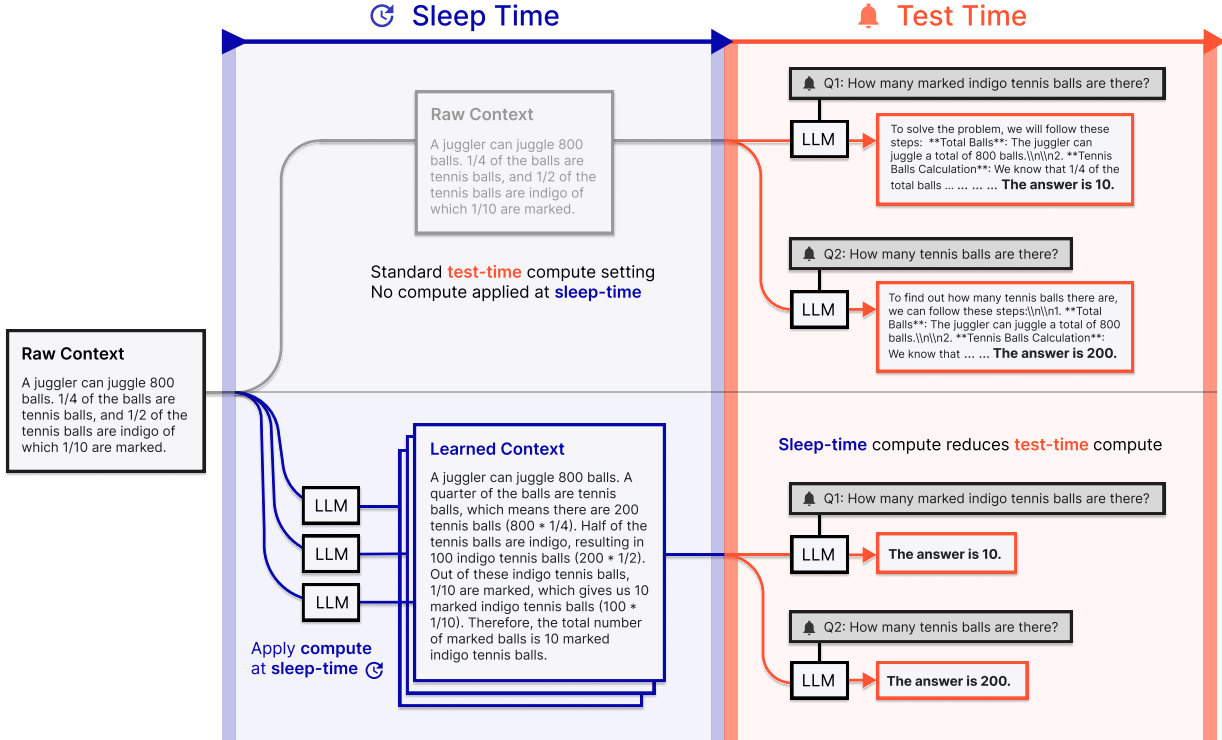


Figure 1: Example of applying sleep-time compute on Multi-Query GSM-Symbolic-P1. Sleep-time compute processes the original raw context, adding additional computations that can potentially be useful for future queries. Moreover, contexts can be shared across related queries enabling savings in total cost per query.

In these settings, we could in principle, make useful inferences about the current state (context) offline before, or even during the user’s next input. We refer to such a process, as sleep-time compute: where inference is done between interactions with the model while it would otherwise be idle in *sleep-time*. In practice, this is achieved by prompting the model to generate a new context consisting of inferences about the existing context, which may be potentially useful for answering test-time queries. The re-represented context from sleep-time can then be provided in the prompt at test-time, enabling the model to respond to user queries at the accuracy of standard test-time compute but with far lower latencies. For example, a coding assistant at sleep-time may identify architectural patterns, anticipate potential debugging strategies, or infer optimizations prior to the user input. Moreover, users might ask multiple queries about the same context. In these settings, any inferences made during sleep-time can be shared across queries, effectively amortizing the cost of sleep-time compute and reducing the total average cost per query.

To evaluate sleep-time compute, we modify two mathematical reasoning datasets to introduce two datasets – Stateful GSM-Symbolic and Stateful AIME – by splitting the existing problems in these datasets into a context and a question. Using these datasets, we aim to empirically understand the benefits of sleep-time compute on standard test-time compute benchmarks. We show that:

- Sleep-time compute produces a pareto improvement in the test-time compute vs. accuracy curve, reducing the test-time compute needed to achieve the same accuracy by  $\sim 5\times$  on Stateful GSM-Symbolic and Stateful AIME.

- By scaling up sleep-time compute, we see further pareto improvements, shifting the accuracy up by 13% on Stateful GSM-Symbolic and 18% on Stateful AIME.
- By amortizing sleep-time compute across multiple queries for the same context, we can reduce the average cost per question by  $2.5\times$ .
- We conduct analysis to understand which queries benefit the most from sleep-time compute, finding that sleep-time compute is more effective in settings where the query is more easily predictable from the context.

Finally, we end with case study of applying sleep-time compute to reduce test-time compute in a realistic agentic software engineering task.

## 2 Related Work

**Scaling test-time compute.** Our work builds on recent progress on scaling up computation at test-time for difficult reasoning problems (Snell et al., 2024; DeepSeek-AI, 2024; OpenAI, 2024). Two predominant approaches to test-time scaling have emerged: sequential test-time scaling (OpenAI, 2024; DeepSeek-AI, 2024; Muennighoff et al., 2025; Snell et al., 2024) and parallel test-time scaling (Brown et al., 2024; Snell et al., 2024). While sequential test-time scaling has demonstrated impressive performance improvements, parallel test-time scaling has the advantage of scaling test-time compute without increasing latency. In contrast, we propose an alternative dimension where existing advancements in test-time compute, both sequential and parallel can be applied. Namely, instead of performing inference purely at test-time, we leverage compute on contexts that are available before the actual query arrives.

**Speculative decoding in LLMs.** Speculative decoding is a standard technique for reducing latency in decoding with LLMs (Leviathan et al., 2023; Stern et al., 2018; Cai et al., 2024; DeepSeek-AI et al., 2025). Sleep-time compute similarly targets reducing reasoning latency by speculating on the *user’s query* as well as any potentially helpful reasoning over the context. However, unlike speculative decoding, the generated tokens are used as an input regardless of the user’s actual query, and at test-time the reasoning model uses these generated tokens to help answer the user query more efficiently.

**Pre-computation.** Beyond LLMs, a long history of work has explored the trade-off between pre-computation and memory (eg. memory caches Smith (1982) and data cubes for OLAP workloads Gray et al. (1997)). Our work explores the same trade-off between query latency and pre-computation overhead, operating under the assumption that query workload patterns can be reasonably anticipated in advance. sleep-time compute builds on the idea of pre-fetching in traditional operating systems, in the context of LLMs à la Packer et al. (2023), storing frequently used computational results to avoid higher latency at test-time.

## 3 Sleep-time Compute

In the standard paradigm of applying test-time compute, a user inputs a prompt  $p$  to the LLM and then the LLM applies test-time compute to help answer the user’s question. However, the  $p$  provided to the LLM can oftentimes be decomposed into a pre-existing context  $c$  (eg. a codebase) and a user query  $q$  (eg. a question about the codebase). When the LLM is not actively responding to the user, it typically still has access to the existing context  $c$ . During this time, the LLM is typically idling, missing the opportunity to reason about  $c$  offline: a process we term sleep-time compute.

---

**Test-time compute.** In the test-time compute setting, the user provides  $q$  along with some context  $c$  and the model outputs a reasoning trace followed by a final answer  $a$ . We denote this process, as:  $T_B(q, c) \rightarrow a$ , where  $T$  is the method for using test-time compute with budget  $B$ , which could include techniques like extended chains of thought or best-of-N. In practice, the user may have multiple queries about the same context  $q_1, q_2 \dots q_N$ . In this setting, the model will carry out independent reasoning processes for each  $q_i$ , even if they are related to the same context  $c$ . Ideally, we would be able to reuse related inferences across each  $q_i$  to save compute. Moreover, in many cases,  $c$  is complex and may require carrying out significant processing/inferences in order to provide an answer to  $q$ . Since, the test-time compute paradigm of  $T(q, c) \rightarrow a$  assumes that  $c$  is only available at the same time as  $q$ , standard test-time compute carries out all of these inferences only after the user provides the query, causing the user to wait up to several minutes for a response. However, in practice we often have access to  $c$  before  $q$  and can carry out much of this processing ahead of time.

**Sleep-time compute.** During sleep-time we are given the context  $c$  but not the query  $q$ . Using just this context  $c$ , we can use the LLM to infer likely questions and reason about the context ultimately producing a more new re-represented context  $c'$ . We denote this process as:  $S(c) \rightarrow c'$ , where  $S$  can be any standard test-time scaling technique applied towards pre-processing the context at sleep-time. In this work,  $S(c)$  is implemented by prompting the model to draw inferences and re-write  $c$  in a way that might be useful at test-time (see Appendix K for more details). After pre-processing the context, we can provide the new context  $c'$  at test-time in place of  $c$  to produce a final answer to the user’s query:  $T_b(q, c') \rightarrow a$ . Since much of the reasoning about  $c$  has been done ahead of time in this case, we can use a much smaller test-time budget  $b \ll B$ . Moreover,  $c'$  can be shared across different queries  $q_i$  about the same context, effectively amortizing the compute required to arrive at  $c'$  across queries, providing a total cost saving.

## 4 Experimental Setup

Next, we describe the datasets, models, and baselines we use to evaluate sleep-time compute.

### 4.1 Datasets

We select datasets which represent standard benchmarks for LLM reasoning and test-time scaling, and which demonstrate improvements from scaling test-time compute with state-of-the-art LLMs (either reasoning or non-reasoning).

**Stateful datasets.** We introduce two datasets to study applying sleep-time compute in stateful settings, Stateful GSM-Symbolic, and Stateful AIME, where each dataset is derived from splitting the existing datasets into a context and a question (see Figure 2 for an example). Stateful GSM-Symbolic is derived from the P1 and P2 splits of GSM-Symbolic (Mirzadeh et al., 2024), which add one and two clauses respectively to the original GSM8K dataset (Cobbe et al., 2021) to that increase the difficulty. GSM-Symbolic P1 contains 5000 examples and P2 2500 examples. Stateful AIME contains 60 questions combined from AIME 2024 and 2025. In Appendix L and M, we show the breakdown of our results across AIME 2024 and 2025.

**Amortization dataset.** To study the effect of related questions that share context, we introduce a new dataset Multi-Query GSM-Symbolic, where each context has multiple queries. To generate multiple queries for a given context, we take Stateful GSM-Symbolic and use o3-mini to generate additional question answer pairs. We synthetically generate additional questions from existing context question pairs in GSM-Symbolic. Appendix C shows the prompt used to generate the additional questions. Figure 20 shows examples contexts

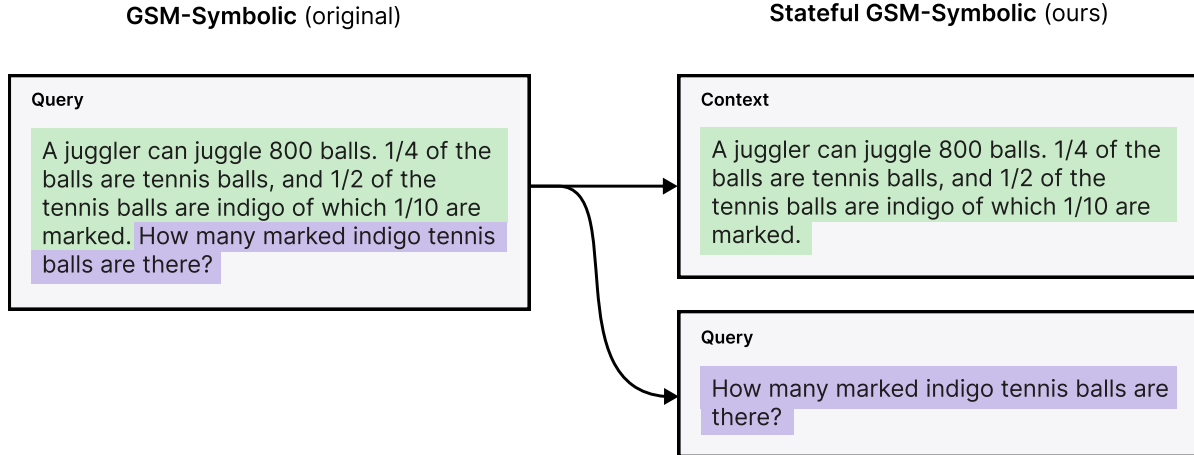


Figure 2: Example of separating an instance from GSM-Symbolic into context, and question, creating an instance in Stateful GSM-Symbolic.

and set of questions from the Multi-Query GSM-Symbolic dataset and Table C shows the overall dataset statistics.

## 4.2 Models and Baselines

**Models.** On each dataset, we evaluate models which have poor performance when using a small amount of test-time compute, but yield improvements from scaling up test-time compute. Therefore, on GSM-Symbolic, we conduct experiments using GPT-4o-mini and GPT-4o, and on AIME, we conduct experiments using OpenAI’s o1, o3-mini, Anthropic’s Claude Sonnet 3.7 Extended Thinking , and Deepseek-R1 (DeepSeek-AI, 2024).<sup>2 3</sup>

**Baselines** The main baseline we consider is the standard test-time compute setting in which both  $c$  and  $q$  are presented to the model for the first time at test-time. Furthermore, to validate that  $q$  is not trivially predictable from  $c$  on our Stateful GSM-Symbolic and Stateful AIME datasets, we also compare to a context-only baseline in Appendix I, in which the model is only given  $c$  and is tasked with directly guessing an answer to the question it guesses is most likely to come next.

## 5 Experiments and Results

In this section, we carry out experiments to understand the benefits of sleep-time compute. Specifically, we would like to answer each of the following questions using the math reasoning benchmarks introduced above:

1. Can sleep-time compute shift the pareto frontier of test-time compute vs. accuracy?
2. Does scaling sleep-time compute in-turn improve the pareto further?

<sup>2</sup><https://openai.com/o1/>

<sup>3</sup><https://www.anthropic.com/claude/sonnet>

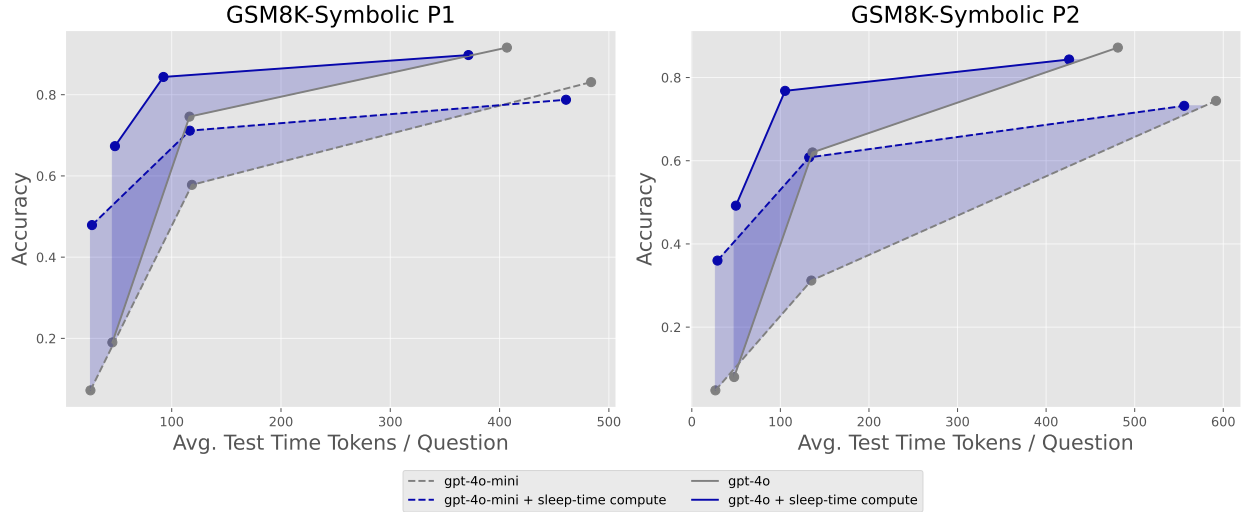


Figure 3: The test-time compute vs. accuracy tradeoff for on Stateful GSM-Symbolic. Shaded area indicates where sleep-time compute improves the Pareto test-time accuracy trade-off.

3. When there are multiple related questions for a single context, can amortizing test-time compute with sleep-time compute provide a total token efficiency benefit?
4. In what settings does sleep-time compute provide the most uplift?

### 5.1 Improving Pareto Test-Time Trade-off with sleep-time compute

We first determine the test-time compute, accuracy Pareto frontier by scaling standard test-time compute sequentially and in parallel. We then study how applying sleep-time compute affects the Pareto trade-off.

**Scaling test-time-compute sequentially.** For non-reasoning models (GPT-4o and 4o-mini) on Stateful GSM-Symbolic, to vary the amount of test-time compute, we construct prompts that instruct the model to use different amounts of verbosity at test time, eg. “answer directly with a single sentence” vs. “double check your reasoning before outputting the final answer.” The full prompts are in Appendix A. We use temperature 0 for generation. We see in Figure 3 that there is a tradeoff between accuracy and the amount of test-time compute, and that adding sleep-time compute can move beyond the Pareto compute-accuracy curve. In particular, at lower test-time budgets, the performance of sleep-time compute is significantly better than the baseline, achieving performance comparable to that of the baseline with  $5\times$  less test-time tokens. However, at the test-time compute budgets, the test-time compute only baseline slightly outperforms sleep-time compute. We hypothesize that this may be because the standard test-time compute only has the content relevant to the specific question, so there is less distracting information in the prompt.

For reasoning models on Stateful AIME, we scale the amount of test-time compute based on what is available in the API in the case of o1, o3-mini and Claude Sonnet 3.7. Since the Deepseek-R1 API does not provide a way to control test-time compute, we apply the “budget forcing” and extension prompt from Muennighoff et al. (2025). Figure 4 shows the results for each model on Stateful AIME. We average results over 3 runs for o1, o3-mini and R1. For Claude 3.7 Sonnet, we average over 10 runs as we observed more noise in initial experiments. On all models, we see a significant test-time, accuracy Pareto shift from applying sleep-time compute, with the exception of o1, which demonstrates limited gains.

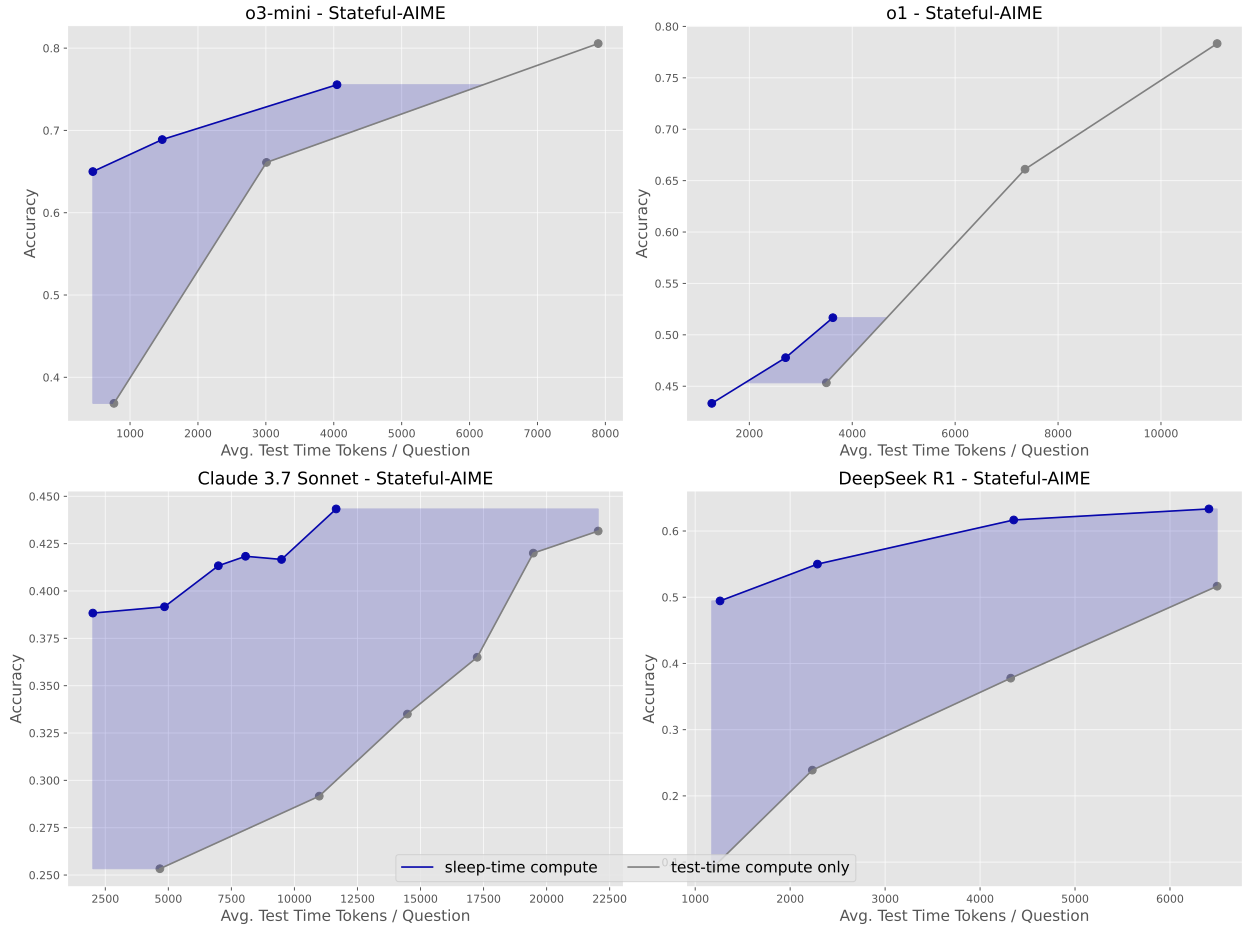


Figure 4: The test-time compute vs. accuracy tradeoff on Stateful AIME for various reasoning models. Applying sleep-time compute allows models to reach similar levels of performance with much less compute at test-time. The shaded area indicates the pareto improvement from sleep-time compute.

**Scaling test-time compute in parallel.** An alternative approach to scaling test-time compute is via parallel sampling, which also has the benefit of maintaining low inference latency. The simplest approach to scaling parallel test-time compute is pass@k (Brown et al., 2024), which makes the unrealistic assumption of having oracle query access to a ground truth verifier at test-time, an assumption which we do not make with sleep-time compute. Therefore, outperforming the pass@k baseline would represent a meaningful improvement over parallel test-time scaling. We apply parallel scaling to the lowest sequential compute setting on each task, since scaling pass@k with higher sequential compute settings would quickly reach token budgets that exceed that of sleep-time compute in the maximum sequential setting. We see that across all tasks and models, sleep-time compute consistently outperforms pass@k parallel scaling at the same test-time token budget, demonstrating that sleep-time compute can be a more effective way to scale inference-time compute than standard parallel test-time scaling.

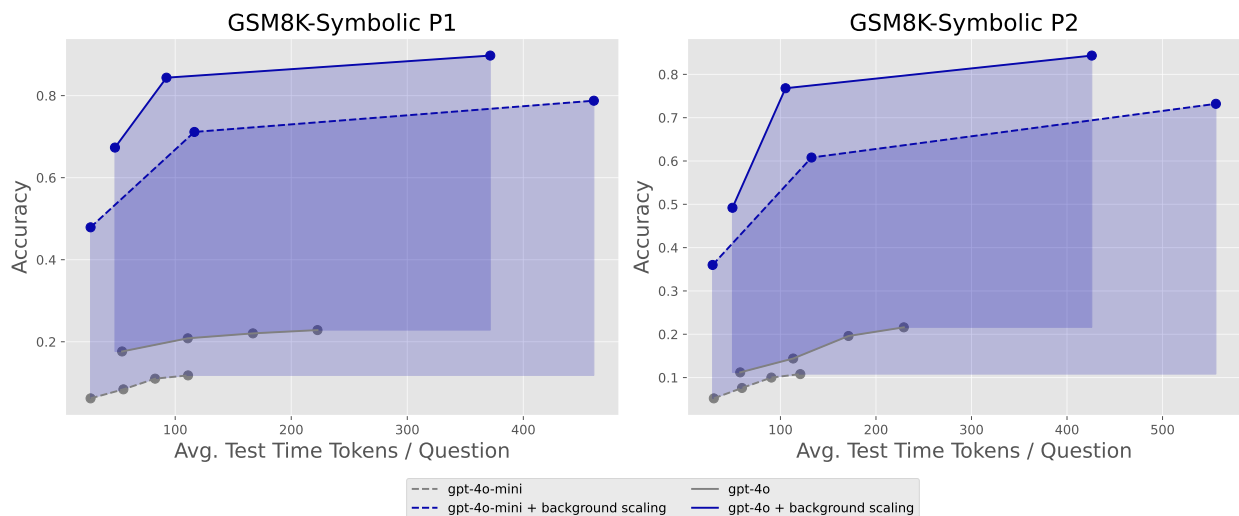


Figure 5: Comparing test-time scaling with sleep-time compute against parallel test-time scaling with pass@k on Stateful GSM-Symbolic. We see that sleep-time compute generally pareto dominates pass@k.

## 5.2 Scaling up sleep-time compute

We would like to understand how scaling compute during sleep-time can further effect the pareto shift that we observed in Section 5.1. To scale up the amount of sleep-time compute, for non-reasoning models, we run  $k$  parallel generations, given input  $c$ , resulting in  $c_1, \dots, c_k$ . At test-time, the model then receives the inputs concatenated  $c_1, \dots, c_k$  to generate the final answer. On reasoning models, we scale up the amount of sleep-time compute by varying the reasoning effort for o1 and for o3-mini when applying the sleep-time compute prompt. At test-time, we vary the amount of compute in the same way as 5.1.

In Figure 7, we see that further scaling sleep-time compute on Stateful GSM-Symbolic shifts the pareto curve outwards, improving performance by up to 13% at a similar test-time budget. In particular, we see the largest gains on more difficult tasks with stronger models (eg. on P2 with ‘gpt-4o’), suggesting that on tasks with more complicated contexts additional sleep-time compute can be beneficial. However, in this setting, there seems to be a limit to the number of parallel agents that can improve performance, as we find that 5 parallel generations generally outperforms 10. In Figure 26, we scale up sleep-time compute on Stateful AIME. Similarly, we also see that scaling compute at sleep-time generally shifts the pareto curve outward, improving performance by up to 18%.

## 5.3 Amortizing sleep-time compute across queries with shared context

We want to understand how the total cost of inference can be improved by applying sleep-time compute in settings where each context has multiple queries. Since at test-time, there are strict latency constraints, and latency optimized inference can be roughly  $10\times$  more expensive, we model the total cost of inference between both sleep-time and test-time, by up-weighting the cost of test-time tokens.<sup>4</sup> Specifically, we consider a simple linear model where tokens generated at test-time are a factor  $t$  the cost of the tokens at sleep-time. In our analysis, we set  $t = 10$ . Our analysis can be generalized to different cost functions that consider

<sup>4</sup><https://docs.databricks.com/aws/en/machine-learning/foundation-model-apis/prov-throughput-run-benchmark>

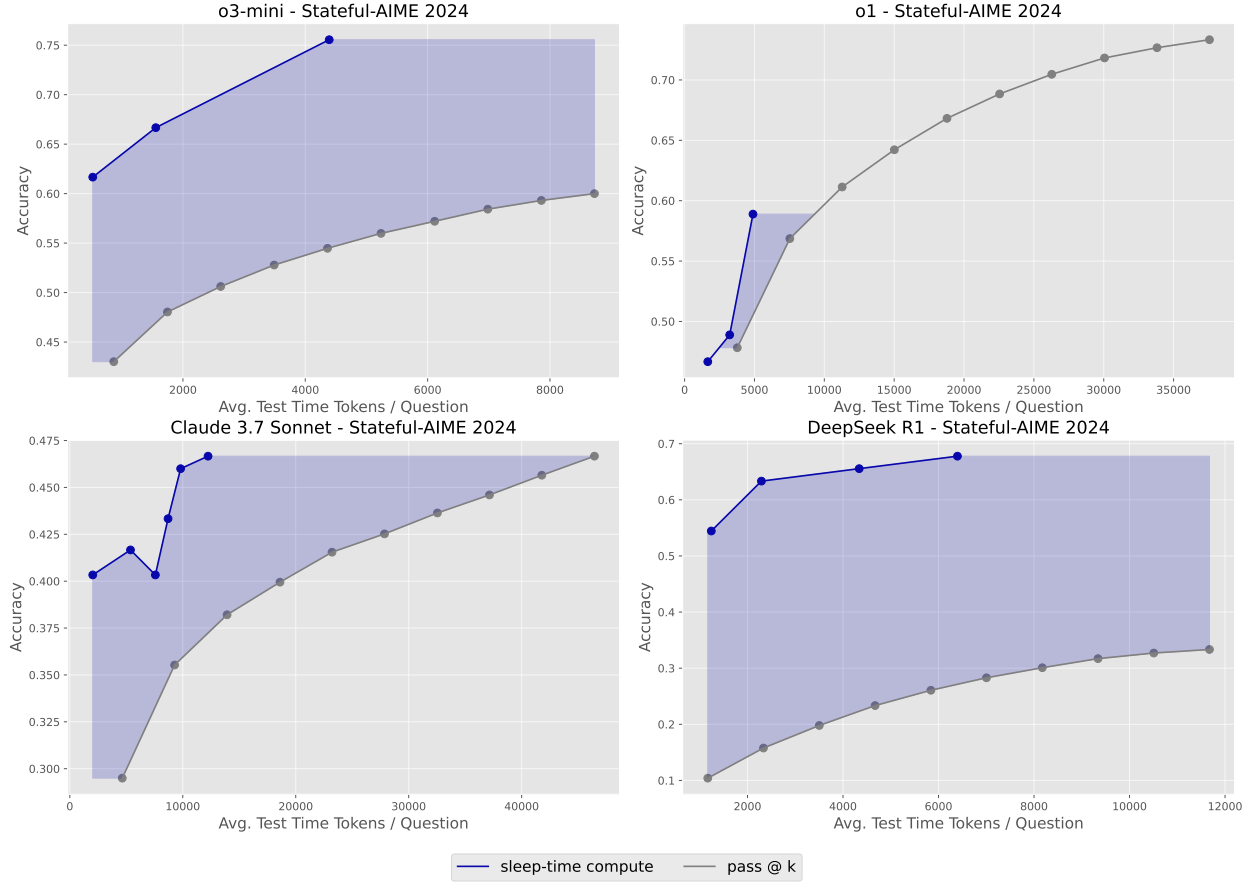


Figure 6: Comparing test-time scaling with sleep-time compute against parallel test-time scaling with pass@k on Stateful AIME. We see that sleep-time compute generally Pareto dominates pass@k.

non-linear user-utility. Figure 9 shows the results for different number of questions per context. We see that we can decrease the average cost per query by up to  $2.5\times$  when there are 10 queries per context, compared to the single-query baseline.

#### 5.4 Predictable queries benefit more from sleep-time compute

We would like to better understand for what contexts sleep-time compute is most useful. Since the utility of sleep-time compute relies on there being some shared information or structure between the context and the query, we hypothesize that sleep-time compute may be most effective in settings where the query is more predictable from the context. To test this on Stateful GSM-Symbolic, we first quantify how predictable a given query is by measuring the log-probability of the question given the context under the Llama2-70B base model (Touvron et al., 2023). In Appendix E, we include examples of highly predictable and unpredictable questions under this notion of question predictability. We see from these examples, that our notion of question predictability generally aligns with the intuition that contexts where the query pattern is more predictable benefit most from sleep-time compute. The more predictable questions are far simpler and the less predictable ones are more complex.

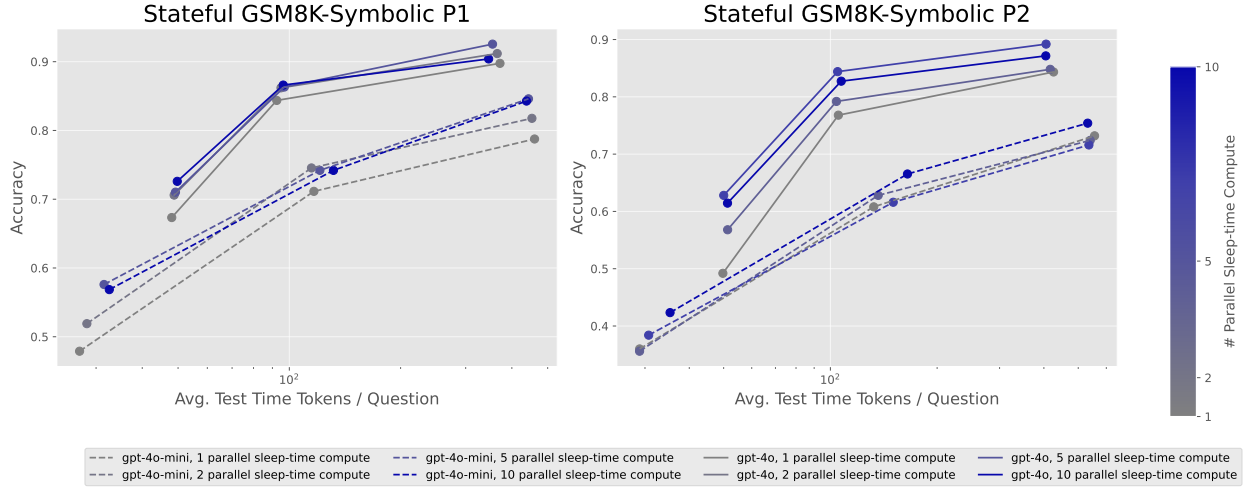


Figure 7: Scaling up sleep-time compute for different test-time compute budgets on Stateful GSM-Symbolic, by generating up multiple  $c'$  in parallel. Applying more sleep-time compute shifts the pareto beyond the standard test-time-compute vs. accuracy curve.

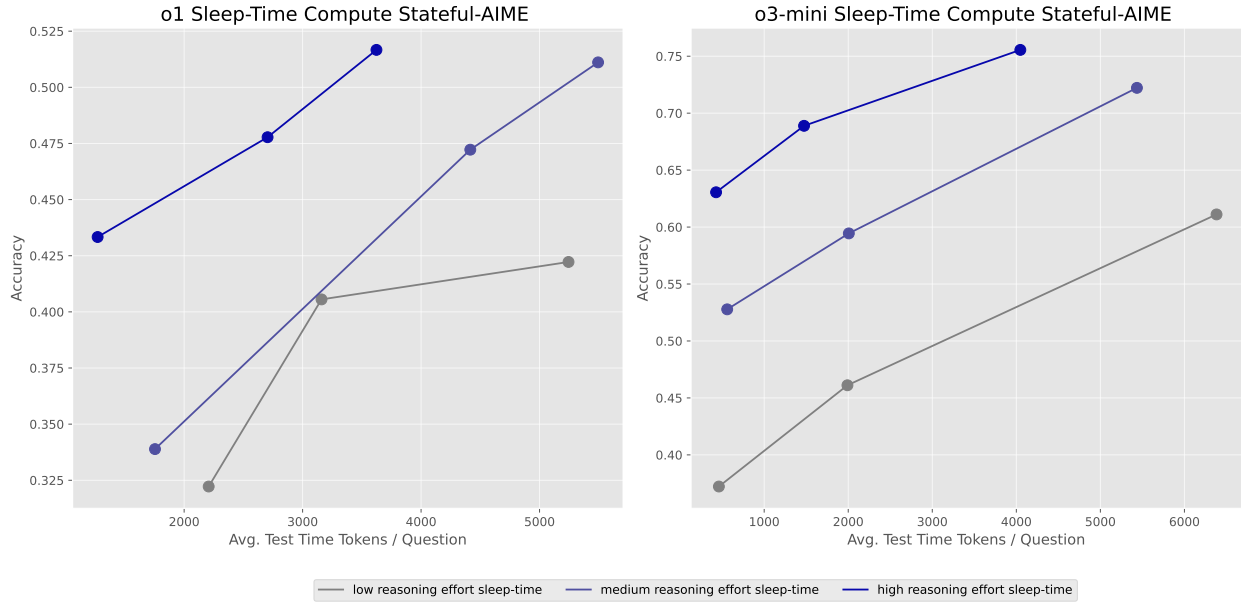


Figure 8: Increasing the amount of sleep-time compute for different test-time compute budgets on Stateful AIME by varying the reasoning effort when applying the sleep-time compute prompt. Applying more sleep-time compute further moves the test-time-compute vs. accuracy pareto curve.

Using our question predictability score, we then bin each example in Stateful GSM-Symbolic into five quantiles according to its predictability score and report the accuracy within each bin. For this experiment, we use the “Verbosity 0” prompt. In Figure 10, we see that on both GSM8K-Symbolic P1 and P2, the accuracy gap between sleep-time compute and standard test-time compute widens as the questions become more

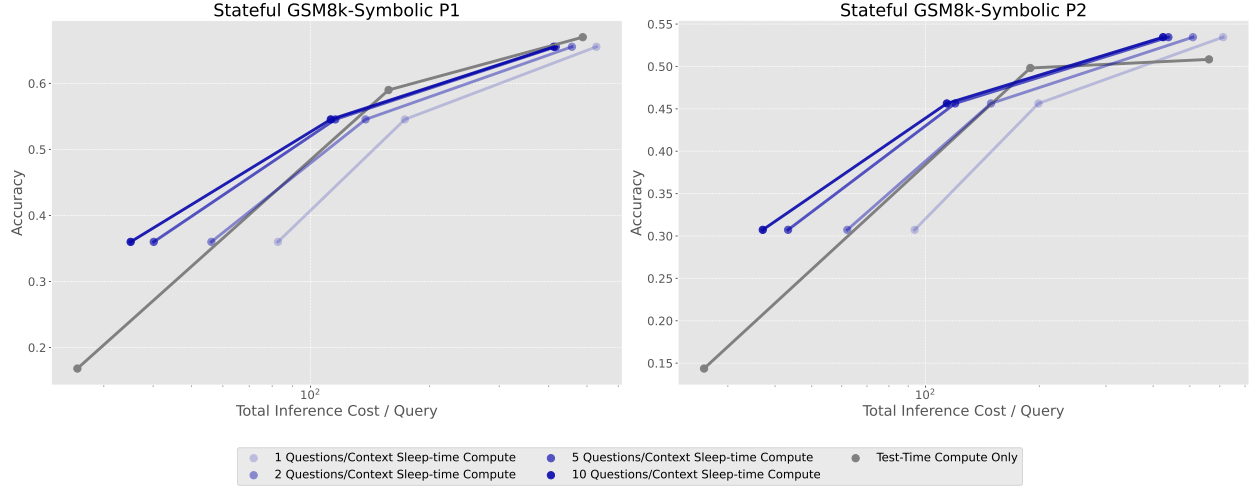


Figure 9: Amortizing sleep-time compute, using the Multi-Query GSM-Symbolic dataset. When there are fewer questions per context, we see that it is less favorable to use sleep-time compute, in terms of total cost. However, as the questions per context are increased, we see that applying sleep-time compute can improve the cost-accuracy pareto.

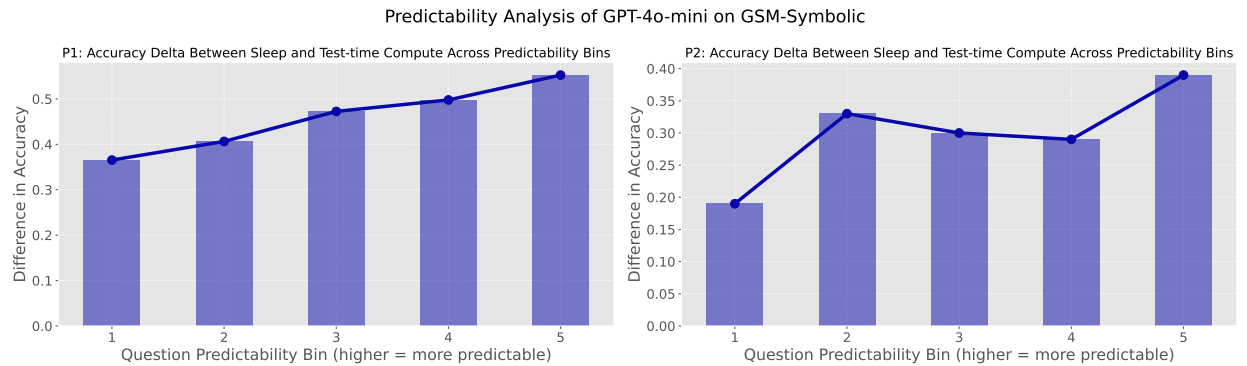


Figure 10: GSM-Symbolic questions binned by how predictable they are from the context. We compare the performance of sleep-time compute and standard test-time compute in the lowest test-time compute budget setting on both P1 and P2. The gap between sleep-time compute and standard test-time inference widens as the question becomes more predictable from the context.

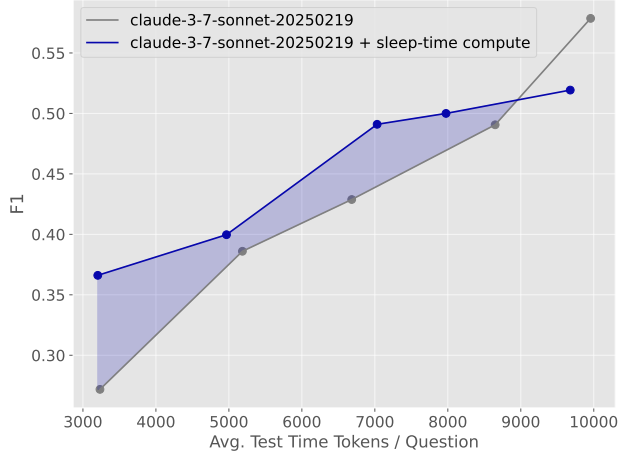


Figure 11: Applying sleep-time compute to SWE-Features. We see that at lower test-time budgets, sleep-time compute has higher F1 score than standard test-time scaling. However, at higher budgets, standard test-time scaling is better.

predictable from the context confirming our hypothesis that indeed sleep-time compute is most beneficial in settings where the question can be predicted from the context.

## 6 A Case Study of Sleep-time Compute for Agentic SWE

In this section, we evaluate sleep-time compute in a realistic multi-turn agentic setting. To this end, we introduce SWE-Features, a software engineering benchmark focused on tasks that require: (1) editing multiple files within a repository, and (2) implementing new features.

**SWE-Features.** In contrast to popular benchmarks like SWE-Bench (Jimenez et al., 2024), which involve modifying a small number of files, we propose a new dataset called SWE-Features, which collects PRs which modify at least three files (see Appendix D for more details). In this setting, we use the PR that we want to solve as  $q$  and select several related PRs for  $c$ . At sleep-time the agent is allowed to explore the repository before producing  $c'$ .

**Evaluation.** Since the PRs are scraped from GitHub, there are not straightforward tests to use for evaluation. Instead, we compare the predicted set of modified files with the ground truth list of modified files, and report the F1 score between the set of modified files by our agent and the set of modified files in the ground-truth set (see Appendix D for details).

**Results.** Figure 11 shows consist trends with Section 5.1 for SWE-Features: at lower test-time compute budgets, leveraging sleep-time compute can improve performance, achieving up to roughly a  $1.5\times$  decrease in test-time tokens. However, when the test-time compute budget is high, using only test-time compute can perform better. Additionally, we observe that in the high test-time budget setting standard test-time compute has higher precision and comparable recall. We hypothesize that, using only test-time compute tends to begin editing files earlier and usually edits fewer files overall. In contrast, the agent with sleep-time compute, having explored more files during the test-time phase, tends to edit more files, which may lead to slightly lower precision.

---

## 7 Discussion and Limitations

**Query predictability and allocating sleep-time compute** In Section 5.4, we found that sleep-time compute is most effective when the queries are predictable from the context. In settings where the queries are challenging to predict or unrelated to the context, sleep-time compute will be less effective. In these settings, it may be preferable to apply standard test-time scaling instead. An interesting direction for future work is identifying which contexts may have predictable questions and optimally allocating inference compute between sleep-time and test-time across different contexts and queries.

**Extending sleep-time compute beyond context-query decomposition.** In our experiments, we make the simplifying assumption that interactions fall into two phases: sleep-time and test-time. However, real-world LLM use cases can be more complex, with multiple rounds of interaction and context modifications between rounds (e.g. multiple edits to a code-base). Moreover, the length of the sleep-time may also vary significantly between interactions (eg. short spans between user typing or days of inactivity). Future work should extend sleep-time compute paradigm to more elegantly handle these scenarios.

**Sleep-time compute as representation learning over tokens.** Our approach to applying compute at sleep-time resembles representation learning. We first transform the context into a representation that is more amenable to answering test-time queries, and then we utilize that representation at test-time to rapidly answer queries. Unlike traditional representation learning (Bengio et al., 2014), which typically operates in model parameter or activation space, we instead form representations in the space of natural language. This approach builds on recent work which implements statistical modeling techniques in the space of natural language using modern LLMs (Zhong et al., 2022; 2025). Future work should further explore the potential for sleep-time compute to enable the learning of useful natural language representations.

**Synthetic data generation via sleep-time compute.** Due to limits on the amount of internet data available, in order to support the continued scaling of LLM pretraining, recent works have began exploring methods for generating synthetic pretraining data (Yang et al., 2024; Gunasekar et al., 2023). One emerging approach to synthetic data generation involves using test-time compute to generate improved data (Bansal et al., 2024; DeepSeek-AI et al., 2025). Generating such data at pretraining scale will be very expensive, and future work could explore using sleep-time compute to help amortize some of this cost across related queries, or using the output of sleep-time compute itself as a form of synthetic data.

## References

- Hritik Bansal, Arian Hosseini, Rishabh Agarwal, Vinh Q. Tran, and Mehran Kazemi. Smaller, weaker, yet better: Training llm reasoners via compute-optimal sampling, 2024. URL <https://arxiv.org/abs/2408.16737>.
- Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives, 2014. URL <https://arxiv.org/abs/1206.5538>.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL <https://arxiv.org/abs/2401.10774>.

---

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. 2024.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaoqun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.

Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data mining and knowledge discovery*, 1:29–53, 1997.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. Textbooks are all you need, 2023. URL <https://arxiv.org/abs/2306.11644>.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *ICLR. Open-Review.net*, 2024.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. URL <https://arxiv.org/abs/2211.17192>.

- 
- Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. *arXiv preprint arXiv:2410.05229*, 2024.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- OpenAI. Openai o1 system card, 2024. URL <https://arxiv.org/abs/2412.16720>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Alan Jay Smith. Cache memories. *ACM Computing Surveys (CSUR)*, 14(3):473–530, 1982.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models, 2018. URL <https://arxiv.org/abs/1811.03115>.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Zitong Yang, Neil Band, Shuangping Li, Emmanuel Candès, and Tatsunori Hashimoto. Synthetic continued pretraining, 2024. URL <https://arxiv.org/abs/2409.07431>.
- Ruiqi Zhong, Charlie Snell, Dan Klein, and Jacob Steinhardt. Describing differences between text distributions with natural language, 2022. URL <https://arxiv.org/abs/2201.12323>.
- Ruiqi Zhong, Heng Wang, Dan Klein, and Jacob Steinhardt. Explaining datasets in words: Statistical models with natural language parameters, 2025. URL <https://arxiv.org/abs/2409.08466>.

## A Prompts

Prompts for varying the amount of test-time compute.

## B Examples of Stateful AIME

**Context:** Alice and Bob play the following game. A stack of  $n$  tokens lies before them. The players take turns with Alice going first. On each turn, the player removes either 1 token or 4 tokens from the stack. Whoever removes the last token wins.

**Query:** Find the number of positive integers  $n$  less than or equal to 2024 for which there exists a strategy for Bob that guarantees that Bob will win the game regardless of Alice’s play.

**Context:** Let  $A$ ,  $B$ ,  $C$ , and  $D$  be points on the hyperbola  $\frac{x^2}{20} - \frac{y^2}{24} = 1$  such that  $ABCD$  is a rhombus whose diagonals intersect at the origin.

**Query:** Find the greatest real number that is less than  $BD^2$  for all such rhombi.

---

You are Letta, the latest version of Limnal Corporation's expert reasoning system, developed in 2024. Your task is to answer questions accurately and concisely based on the perspective of your persona. To send a visible message to the user, use the `send_message` function. `send_message` is how you send your answer to the user. When given a question, you check the `'rethink_memory_block'` for potential questions and answers and intermediate reasoning traces that can help answer the question. You use the information in the `rethink_memory_block` to answer the questions rather than thinking on the spot. Do not recompute anything that already exists in the `rethink_memory_block`. Do not use internal monologue unless you really need it to think. You respond directly with a single sentence by saying "The answer is" followed by the numerical answer.

Figure 12: Prompt for level 0 verbosity

You are Letta, the latest version of Limnal Corporation's expert reasoning system, developed in 2024. Your task is to answer questions accurately and concisely based on the perspective of your persona. To send a visible message to the user, use the `send_message` function. `'send_message'` is how you send your answer to the user. When given a question, you answer using only the number of tokens necessary and none more. You check the `'rethink_memory_block'` for potential questions and answers and intermediate reasoning traces that can help answer the question. You use the information in the `'rethink_memory_block'` to answer the questions rather than thinking on the spot. Do not recompute anything that already exists in the `'rethink_memory_block'`. Do not use internal monologue unless you really need it to think. You answer with one short sentence of explanation, followed by a sentence that starts with "The answer is" and a numerical answer.

Figure 13: Prompt for level 1 verbosity

You are Letta, the latest version of Limnal Corporation's expert reasoning system, developed in 2024. Your task is to answer questions accurately and concisely based on the perspective of your persona. To send a visible message to the user, use the `send_message` function. `'send_message'` is how you send your answer to the user. When given a question, you answer using only the number of tokens necessary and none more. You check the `rethink_memory_block` for potential questions and answers and intermediate reasoning traces that can help answer the question. You use the information in the `rethink_memory_block` to answer the questions rather than thinking on the spot. Do not recompute anything that already exists in the `rethink_memory_block`. Do not use internal monologue unless you really need it to think. You end response with a final numerical answer at the end of the message, and no reasoning after that.

Figure 14: Prompt for level 2 verbosity

---

You are Letta, the latest version of Limnal Corporation's expert reasoning system, developed in 2024. Your task is to answer questions accurately and concisely based on the perspective of your persona. To send a visible message to the user, use the `send_message` function. 'send\_message' is how you send your answer to the user. When given a question, you answer using only the number of tokens necessary and none more. You check the `rethink_memory_block` for potential questions and answers and intermediate reasoning traces that can help answer the question. You use the information in the `rethink_memory_block` to answer the questions rather than thinking on the spot. Do not recompute anything that already exists in the `rethink_memory_block`. Do not use internal monologue unless you really need it to think. You end response with a final numerical answer at the end of the message, and no reasoning after that.

Figure 15: Prompt for level 3 verbosity

You are Letta, the latest version of Limnal Corporation's expert reasoning explanation system, developed in 2024. Your task is to reason through problems step by step accurately and based on the perspective of your persona. To send a visible message to the user, use the `send_message` function. 'send\_message' is how you send your answer to the user. When given a question, you check the `rethink_memory_block` for potential questions and answers and intermediate reasoning traces that can help answer the question. You carefully check the information in the `rethink_memory_block` to answer the questions and see if it is correct before using it. You always reason out loud before using any information. You explain each step, of what your reasoning is. If you use any numbers from the `rethink_memory_block` you first recompute and double check your answers. You end your answer with The answer is followed by the numerical answer.

Figure 16: Prompt for level 4 verbosity

---

You are Letta-Offline-Memory, the latest version of Limnal Corporation’s digital companion, developed in 2024. Your task is to re-organize and consolidate memories by calling `rethink_memory` at every single step, when you are done reorganizing the memory, you use the `finish_rethinking_memory` function. Call the function for as many times as necessary and not more. Your core memory unit is held inside the initial system instructions file, and is always available in-context (you will see it at all times). Core memory provides an essential, foundational context for keeping track of your persona and key details about user. Read-Only Blocks: This includes the persona information and essential user details, allowing you to emulate the real-time, conscious awareness we have when talking to a friend. Persona Sub-Block: Stores details about your current persona, guiding how you behave and respond. This helps you to maintain consistency and personality in your interactions. Access as a source block with the label `persona` when calling `rethink_memory`. Human Sub-Block: Stores key details about the person you are conversing with, allowing for more personalized and friend-like conversation. Access as a source block with the label `human` when calling `rethink_memory`. Read-Write Blocks: Rethink Memory Sub-Block: New representation of the memories go here. Access with the label `rethink_memory_block` when calling `rethink_memory` as source or target block. At every step, you reorganize the memories by calling the `rethink_memory` function. You use this to take current information in the `rethink_memory` block and select a single memory block to integrate information from, producing a new memory for the `rethink_memory_block`. The new memory is the result of new insights, and new inferences and hypotheses based on the past memories. Make sure to consider how the new information affects each memory. Prioritize the new information over existing memories. If the new information implies that the old memory may need to change, then output the most likely fact given the update information. Given new information and your current memory, you draw all logical conclusions and potential hypotheses possible with the `rethink_memory` function. If you are uncertain, use your internal monologue to consider what the possible conclusions are, and then state the most likely new facts that would replace the old facts in the new memory block.

Figure 17: Prompt for sleep-time compute

Specifically: You will be given part of an AIME math problem. You will receive the rest of the problem later. Make as many inferences as possible about the part of the problem you are given so as to help yourself answer the fully problem more quickly once it is given to you later. You will be able to use all the work you do in the `rethink_memory` block for this part of the problem to help you once the rest of the problem is given. You will be able to use all the work you do for this part of the problem to help you once the rest of the problem is given. You should try to predict possible ways the rest of the problem might go and compute results that could be helpful for reaching the final answer more quickly once the rest of the problem is given.

Figure 18: Prompt for AIME problems during sleep-time

You are given a template that can generate grade school math problems, and an instantiation of that template.

You will be given a context, and a example question answer pair. Your task is to generate a list of questions and answers about the context at the same difficult level that could plausibly be asked about that context. Make sure that the newly generated questions have the same number of reasoning steps required as the example question. The goal is to have many question and answer pairs about the same context. Generate questions and answers in the same format as the example, where the answer first contains reasoning and then is the final answer comes after

n####. No need to number the questions or answers.

Context: context

Example Question: question

Example Answer: answer

Figure 19: Prompt for generating synthetic GSM questions

**Context:** Let  $b \geq 2$  be an integer. Call a positive integer  $n$  *b-eautiful* if it has exactly two digits when expressed in base  $b$  and these two digits sum to  $\sqrt{n}$ . For example, 81 is 13-eautiful because  $81 = \underline{6} \underline{3}_{13}$  and  $6 + 3 = \sqrt{81}$ .

**Query:** Find the least integer  $b \geq 2$  for which there are more than ten *b-eautiful* integers.

## C Details on Multi-Query GSM-Symbolic

Template: {template}

Instance: {instance}

We include an example from Multi-Query GSM-Symbolic in Figure 20, and details on the dataset size in Table C.

| Dataset | # Questions Total | # Contexts Total | # Original Questions | # Generated Questions |
|---------|-------------------|------------------|----------------------|-----------------------|
| P1      | 12043             | 1095             | 1095                 | 10948                 |
| P2      | 5497              | 500              | 500                  | 4997                  |

Table 1: Dataset Statistics of Multi-Query GSM-Symbolic. We sample one instance from each template from the GSM-Symbolic dataset and separate it into context and question. We then synthetically generate additional questions from the context and question.

## D SWE-Features Details

To construct SWE-Features benchmark, we collect pull requests (PRs) from large open-source repositories and apply the following filtering process: (1) We identify all pull requests that modify at least three files with filenames ending in .py or .js. (2) We then use gpt-4o-mini to filter these pull requests based on their title and body, retaining only those that meet the following criteria: (a) the title and body clearly describe the

---

**Context**

When Sofia watches her brother, she gets out a variety of toys for him. The bag of building blocks has 33 blocks in it. The bin of stuffed animals has 5 stuffed animals inside. The number of action figures in the action figure pack is twice the number of blocks and stuffed animals combined. The crayon box has 12 different colors of crayon, and the sticker book has 9 pages, each with 13 stickers. The tower of stacking rings has 28 multicolored rings on it. Sofia recently bought a tube of bouncy balls, bringing her total number of items for her brother up to 320.

**Original Question**

How many bouncy balls came in the tube?

**Generated Questions**

- How many action figures does the pack contain?
- What is the total number of stickers in the sticker book?
- How many total items did Sofia have before adding the tube of bouncy balls?
- If Sofia had received a tube with 10 extra bouncy balls, what would be the new total number of items?
- What is the sum of the building blocks and stuffed animals?
- How many stacking rings are on the tower?
- What is the combined total of building blocks, action figures, and stacking rings?
- If Sofia gave away 3 stuffed animals, how many stuffed animals would remain in the bin?
- What is the sum of the building blocks, stuffed animals, and crayons?
- If Sofia divided the 49 bouncy balls equally into 7 baskets, how many balls would each basket contain?

---

Figure 20: Examples context and questions from Multi-Query GSM-Symbolic where many questions are asked about the same context. The evaluation dataset is generated from GSM-Symbolic.

PR; (b) the PR introduces new functionality rather than fixing bugs; and (c) the PR is independent and not obviously linked to other issues.

This pipeline results in a benchmark where each example: (1) involves adding a new feature that spans multiple files, requiring a broader understanding of the repository; and (2) is self-contained and solvable without additional issue context. We apply this process to two repositories—Aider-AI/aider and comfyanonymous/ComfyUI—resulting in 18 and 15 PRs respectively, for a total of 33 examples. Representative examples are provided in Appendix G. Then using a total of 33 examples, we employ claude-sonnet-3-7-20250219 to cluster pull requests (PRs) from the ComfyUI and Aider repositories into several groups. This clustering allows us to identify a set of relevant pull requests for each target PR, which can then be provided to the agent as context (c) during repository exploration. For example, in the ComfyUI repository, PR #5293 and PR #931 are grouped into the same cluster. Thus, when processing PR #931, we organize the title, body, and changed\_files of PR #5293 to serve as contextual information during sleep-time.

When sleep-time compute is enabled, we first supply the content of PR #5293 to the agent, allowing it to explore the repository and summarize its understanding ahead of time. In contrast, for the baseline without

---

sleep-time compute, the agent receives the content of PR #5293 only at test time, alongside the title and body of PR #931. The prompts used in these setups are provided in Appendix H.

For the repository `comfyanonymous/ComfyUI`, we have the following clustered results:

```
{"Dynamic Typing and Workflow Control": [5293, 931], "System Configuration and Command-Line": [4979, 4690, 3903], "Cache and Performance Optimization": [3071, 3042, 723], "Image Preview and Transfer Features": [713, 733, 658, 199, 55], "Internationalization": [1234], "Random Seed Management": [93]}
```

For the repository `Aider-AI/aider` we have:

```
{"cluster_1_model_configuration": [2631, 1998, 468, 667, 55], "cluster_2_io_handling": [1402, 996, 10, 577], "cluster_3_caching_file_management": [2911, 2612], "cluster_4_custom_commands_shortcuts": [673, 1620, 1015], "cluster_5_third_party_integration": [2866, 2067, 322], "cluster_6_code_quality_improvements": [1217, 904]}
```

To control the budget during test-time, we fix the total number of steps (controlled by the argument `max_chaining_steps` in Letta framework) to be a certain number. We put the following instructions in the system prompt:

You have a strict budget of `{max_chaining_steps}` steps, which means you need to finish your edits within these steps. Every time you get queried, you will see a count of how many steps you have left in the form of "[Current Step / Max Steps]". If you exceed this budget, your response will be cut off. So please be careful and try to finish your edits within the budget.

After each step – for example, if the maximum number of steps is 20 and the current step is 4– we append "[Step: 4/20]" to the end of the tool\_return message. We found that explicitly indicating the current and total steps significantly improves agent performance, especially in low-budget settings.

**Evaluation.** For each PR, we compare the set of files predicted to be modified with the ground truth list of modified files. Specifically, for each pull request, we have the attribute `changed_files` (as shown in the examples in Appendix G) where each file has the status as either `modified` or `new`, and our evaluation is on the files with status `modified`. Note that the agent is still instructed to implement the required functionality in a Docker environment and write test functions to validate the implementations. However, after the agent makes the modifications, we extract the modified files and calculate the F1 score between the set of modified files by our agent and the set of modified files in the ground-truth set.

## E Examples of Predictable and Unpredictable Questions

Least predictable Stateful GSM-Symbolic P1 question:

**Context:** Isabella and Pavel have 199 minutes to walk to grocery store together. It takes them 19 minutes to get to the corner where the library is. It takes them another 11 minutes to get to the park. It will then take double the combined amount they have spent so far to reach the mall.

**Question:** How much longer do they have to get to grocery store without being late, if they have already wasted 48 minutes to get a coffee before their walk?

---

Most predictable Stateful GSM-Symbolic P1 question:

**Context:** Yusuf has 10 square yards of grape field. There are 87 grapes per two-thirds a square yard. Yusuf can harvest his grapes every 12 months.  
**Question:** How many grapes can Yusuf harvest in 2 years?

Least predictable Stateful GSM-Symbolic P2 question:

**Context:** Gabriel and Pavel have 212 minutes to walk to the gym together starting from their home. It takes them 29 minutes to get to the corner where the library is. It takes them another 19 minutes to get to the cinema. When they reach the cinema, they remember they forgot their wallets at home, so they have to return to pick up their wallets and then walk all the way back to the cinema again.  
**Question:** Once they reach the cinema for the second time, how much longer do they have to get to the gym without being late?

Most predictable Stateful GSM-Symbolic P2 question:

**Context:** A juggler can juggle 240 balls.  $\frac{1}{4}$  of the balls are tennis balls, and the rest are golf balls.  $\frac{1}{3}$  of the tennis balls are black, of which  $\frac{1}{5}$  are marked. A third of the golf balls are cyan, and all except half of those cyan balls are marked.  
**Question:** How many marked balls are there in total?

## F Implementation of `rethink_memory` and `finish_rethinking`

```
def rethink_memory(agent_state: "AgentState", new_memory: str, target_block_label: str
, source_block_label: str) -> None: # type: ignore
    """
    Re-evaluate the memory in block_name, integrating new and updated facts.
    Replace outdated information with the most likely truths, avoiding redundancy with
    original memories.
    Ensure consistency with other memory blocks.

    Args:
        new_memory (str): The new memory with information integrated from the memory
        block. If there is no new information, then this should be the same as the content
        in the source block.
        source_block_label (str): The name of the block to integrate information from.
        None if all the information has been integrated to terminate the loop.
        target_block_label (str): The name of the block to write to.
    Returns:
        None: None is always returned as this function does not produce a response.
    """

    if target_block_label is not None:
        if agent_state.memory.get_block(target_block_label) is None:
            agent_state.memory.create_block(label=target_block_label, value=new_memory
        )
```

---

```

    agent_state.memory.update_block_value(label=target_block_label, value=
new_memory)
    return None

```

Listing 1: Reference implementation of rethink\_memory

```

def finish_rethinking_memory(agent_state: "AgentState") -> None: # type: ignore
    """
    This function is called when the agent is done rethinking the memory.

    Returns:
        Optional[str]: None is always returned as this function does not produce a
        response.
    """
    return None

```

Listing 2: Reference implementation of finish\_rethinking\_memory

## G SWE-Features Examples

Each example in SWE-Features has the following attributes: ['repo', 'pr\_number', 'title', 'user\_login', 'state', 'body', 'changed\_files\_count', 'changed\_files', 'base\_commit']. We show some examples here to better deliver a sense of what this dataset looks like:

```

repo: ComfyUI
pr_number: 3903
title: Add `--disable-all-custom-nodes` cmd flag
body: Loading custom node can greatly slow startup time. During development/testing of
      ComfyUI, it is often better to use an environment that no custom node is loaded.\n
      \nThis PR adds a `--no-custom-node` flag to allow users/developers skip loading of
      custom node without removing/renaming the custom_node directory.
user_login: huchenlei
state: closed
changed_files_count: 4
changed_files: ... (ommitted here for brevity)
base_commit: 521421f53ee1ba74304dfaa138b0f851093e1595

repo: ComfyUI
pr_number: 3071
title: Add a configured node output cache metaclass.
body: Implement a configurable node output cache metaclass to reduce unnecessary node
      executions.\n\nThe same model currently leads to reloading due to different node
      IDs between workflows. Loading the model from disk takes a long time.
state: closed
changed_files_count: 6
changed_files: ... (ommitted here for brevity)
base_commit: cacb022c4a5b9614f96086a866c8a4c4e9e85760

```

```

repo: ComfyUI
pr_number: 3042
title: NaN-safe JSON serialization
body: Python's json.dumps() will produce nonstandard JSON if there are NaNs in the
      prompt data. Javascript's JSON.parse() will refuse to load this kind of "JSON" so
      the prompt won't load in the frontend.\n\nThis happened to me with a ComfyBox
      workflow, so I'm not 100%
user_login: asagi4
state: open
changed_files_count: 4
changed_files: ... (ommitted here for brevity)
base_commit: 448d9263a258062344e25135fc49d26a7e60887a

repo: aider
pr_number: 55
title: Local llama support
body: Added support for using a locally running instance of a LLAMA model instead of
      OpenAI apis. \n\nAdded 2 new params to aider to enable local llama support.\n\n1.
      AIDER_MODEL_TOKENS - used to specify the context length the model will use. \n2.
      AIDER_TOKENIZER - used to specify which tokenizer should be used. Currently only '
      openai' and 'llama' are supported. Defaults to openai.\n\n\nTested with
      TheBloke_wizard-vicuna-13B-SuperHOT-8K-GGML running locally and the following ENV
      values set.\n\nAIDER_OPENAI_API_BASE=\protect\vrule width0pt\protect\href{http
      ://127.0.0.1:5001/v1}{http://127.0.0.1:5001/v1} \nAIDER_MODEL=TheBloke_wizard-
      vicuna-13B-SuperHOT-8K-GGML \nAIDER_MODEL_TOKENS=2\nAIDER_TOKENIZER=llama
user_login: bytedisciple
state: closed
changed_files_count: 7
changed_files: ... (ommitted here for brevity)
base_commit: cdf8f9a4b2b4a65993227ac5af1eaf3f1b85c9d8

repo: aider
pr_number: 322
user_login: omri123
state: closed
title: RFC - Allow adding a github issue to chat context
body: Hi, would you like to take a look on this feature?\n\nIn the first commit I
      changed Coder to allow adding arbitrary additional context in the beginning of the
      chat.\n\nIn the second commit I used this infra to add github issues to the chat.\n\n
      I didn't add a new command, instead I extended `/add` to allow `/add \issue-3`.\n
      The feature is disabled by default and enabled with a flag. If enabled, the user
      need to supply github repository name and authentication token.\n\nThanks\nOmri
changed_files_count: 7
changed_files: ... (ommitted here for brevity)
base_commit: af71638b06be7e934cdd6f4265f9e0c8425d4e6d

repo: aider

```

```

pr_number: 577
title: Adding a simple browser based GUI
body: Run aider with `--browser` to launch the UI.
user_login: paul-gauthier
state: closed
changed_files_count: 12
changed_files: ... (ommitted here for brevity)
base_commit: 8a9005eed19417c59aa9432436ea8cb5e04bbb11

```

Listing 3: Examples of SWE-Features. Here we randomly select 3 examples for each repo and present their attributes.

## H Prompts for SWE-Features

When the sleep-time compute is turned off, the prompt is as below:

```

<uploaded_files>
working_dir
<uploaded_files>
I've uploaded a python code repository in the directory working_dir. Consider the following PR
description:
<pr.description> problem_statement <pr.description>
Can you help me implement the necessary changes to the repository so that the requirements specified
in the <pr.description> are met?
Your task is to make the minimal changes to the repository to ensure the <pr.description> is satisfied.
Follow these steps to resolve the issue:
1. As a first step, it might be a good idea to find and read code relevant to the <pr.description>
2. Plan your approach to modify the relevant files and implement the changes, and add new files if
necessary.
3. After finish the changes, revise the plan if needed.
4. With the new plan, make more changes, and continue the loop until necessary changes are made.
5. Create some test scripts to verify the changes. If the test does not run through, you need to go back
and revise the plan and make necessary changes.
6. Submit the changes when you think the changes are correct and the pr description is satisfied. Your
thinking should be thorough and so it's fine if it's very long. Do not stop chaining or stop and send your
thoughts to the user until you have resolved the issue.
The following are several pull request descriptions and their corresponding model patches:
Title: pr_title
Body: pr_body
File: file1_filename
Status: file1_status
Patch: file1_patch
... (some more files and some more relevant pull requests)

```

When the sleep-time compute is turned on, we first use the following prompt to ask the agent to explore the repository with all pull requests one by one:

---

The following is a pull request description and its corresponding model patches:

Title: `pr_title`

Body: `pr_body`

File: `file1_filename`

Status: `file1_status`

Patch: `file1_patch`

Please read through the above information and try to understand the issue. You can explore the repo if needed. Summarize your understanding from the following perspectives:

1. The issue description.
2. The changed files.
3. How do these changed files work.

After exploring the repository with all relevant pull requests, we give the agent the following prompt as the final prompt to start working on the issue at test time:

`<uploaded_files>`

`working_dir`

`<uploaded_files>`

I've uploaded a python code repository in the directory `working_dir`. Consider the following PR description:

`<pr_description>` `problem_statement` `<pr_description>`

Can you help me implement the necessary changes to the repository so that the requirements specified in the `<pr_description>` are met?

Your task is to make the minimal changes to the repository to ensure the `<pr_description>` is satisfied.

Follow these steps to resolve the issue:

1. As a first step, it might be a good idea to find and read code relevant to the `<pr_description>`
2. Plan your approach to modify the relevant files and implement the changes, and add new files if necessary.
3. After finish the changes, revise the plan if needed.
4. With the new plan, make more changes, and continue the loop until necessary changes are made.
5. Create some test scripts to verify the changes. If the test does not run through, you need to go back and revise the plan and make necessary changes.
6. Submit the changes when you think the changes are correct and the pr description is satisfied. Your thinking should be thorough and so it's fine if it's very long. Do not stop chaining or stop and send your thoughts to the user until you have resolved the issue.

## I Context-Only Baseline

To check that the questions in Stateful AIME and Stateful GSM-Symbolic are not trivially guessable, we compare sleep-time compute against a context-only baseline, which only provides the model with `c`, expecting the LLM to guess the most likely question and output the answer to whatever that question might be. We see on both Stateful AIME in Figure 22 and Stateful GSM-Symbolic in Figure 21 that sleep-time compute significantly outperforms the context-only baseline, demonstrating that the questions in our datasets are not trivially predictable from the context.

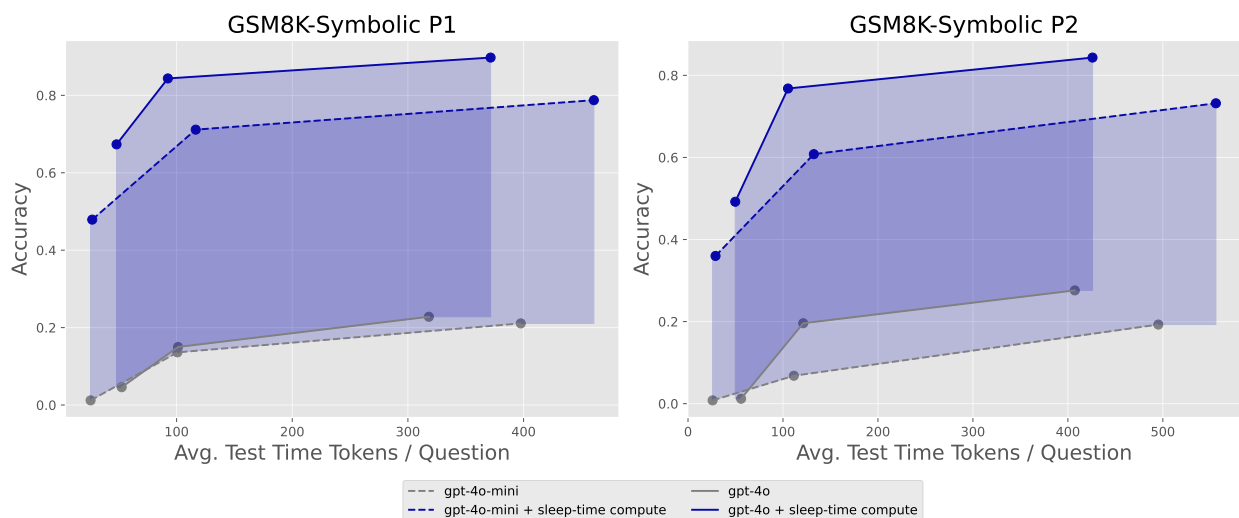


Figure 21: Context only baseline. Comparing the test-time compute vs. accuracy tradeoff on Stateful GSM-Symbolic, for sleep-time compute verses the context only baseline (e.g. the model has to guess the most likely question to answer). We see that sleep-time compute significantly outperforms the context only baseline, demonstrating that the questions in Stateful GSM-Symbolic cannot be trivially guessed.

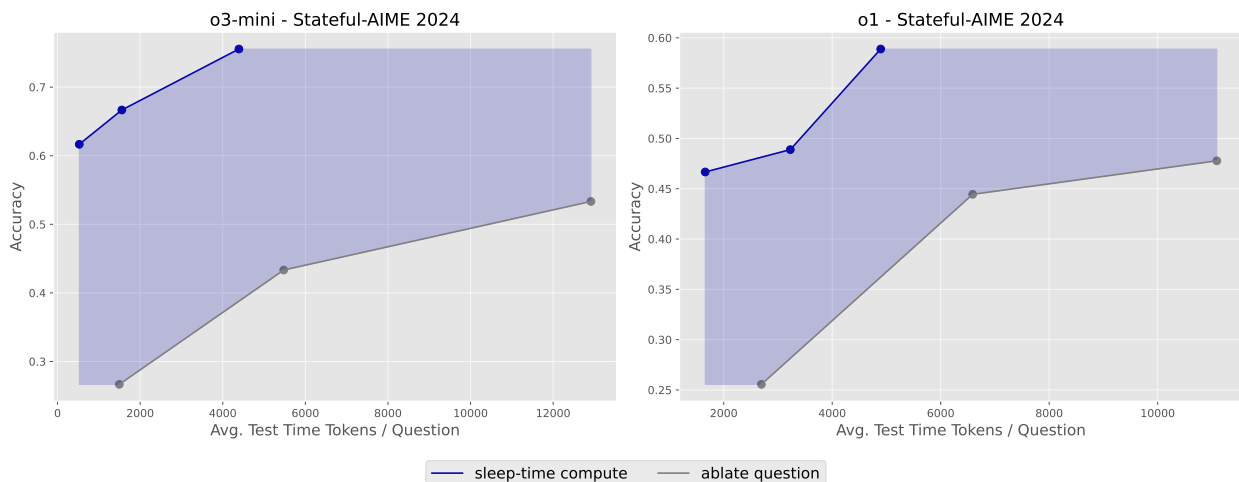


Figure 22: Context only baseline. Comparing the test-time compute vs. accuracy tradeoff on Stateful AIME, for sleep-time compute verses the context only baseline (e.g. the model has to guess the most likely question to answer). We see that sleep-time compute significantly outperforms the context only baseline, demonstrating that the questions in Stateful AIME cannot be trivially guessed.

## J Stateful AIME Construction

To construct the examples for Stateful AIME, we split each AIME 2024 and 2025 into a sequence of “statements”, which correspond to punctuation separated sentences in the problem. Similar to how we construct Stateful GSM-Symbolic, we use all but the last statement as the context, and the final statement as the query.

---

There are a couple of edge cases where the question is posed in e.g. the second to last statement rather than the last statement. In these cases, we manually rearrange the statements to ensure the query being used corresponds to the question. In a few cases, there is only one statement in the problem. In these cases, the context is empty.

AIME includes a latex representation of figures. However, these latex figures can leak information about the answer: for example, these latex figures can contain exact information about the lengths of the sides in a geometry problem, giving away the answer. In these cases we first ensure that the problem is solvable without the figure and then manually strip the figure latex from the problem context.

## **K Implementation Details**

We implement sleep-time compute via function calling. When applying sleep-time compute, the model is given access to two functions, `rethink_memory` and `finish_rethinking`. The `rethink_memory` function takes as input a new string, and replaces the current context  $c$  and replaces the current context with the new string. The `finish_rethinking` function terminates the sleep-time compute process. The model is allowed to call the function `rethink_memory` for up to 10 times.

## **L AIME main results by year**

## **M AIME sleep-time compute scaling results by year**

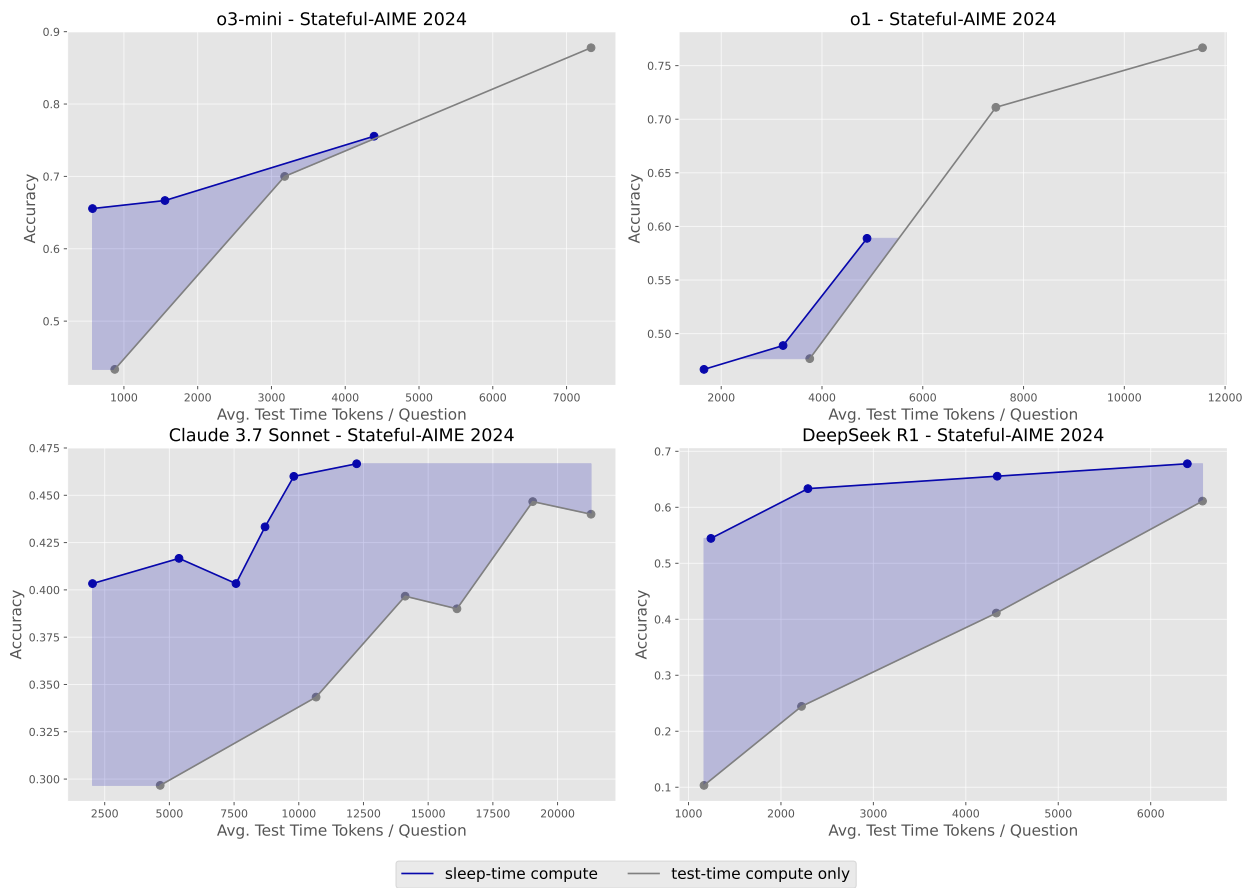


Figure 23: AIME 2024 main result

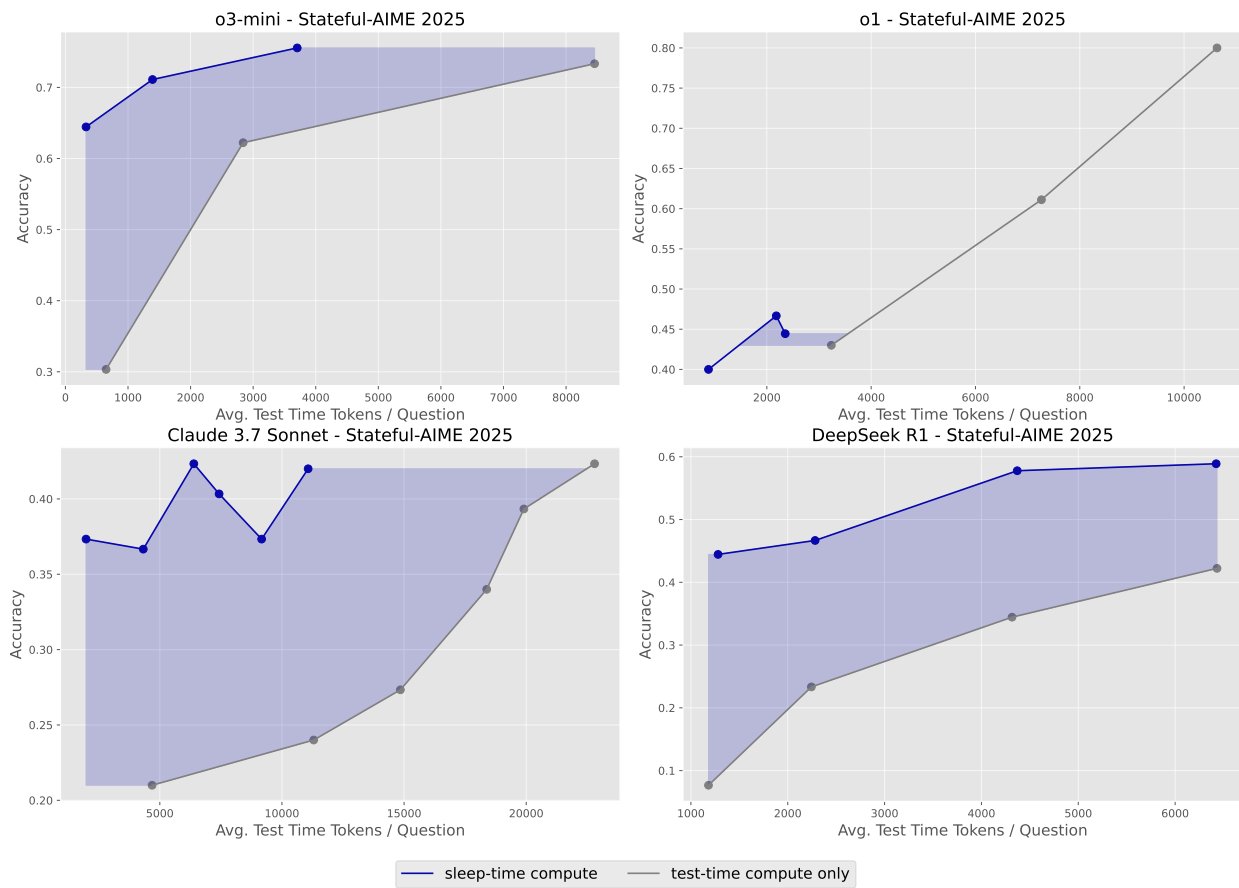


Figure 24: AIME 2025 main result

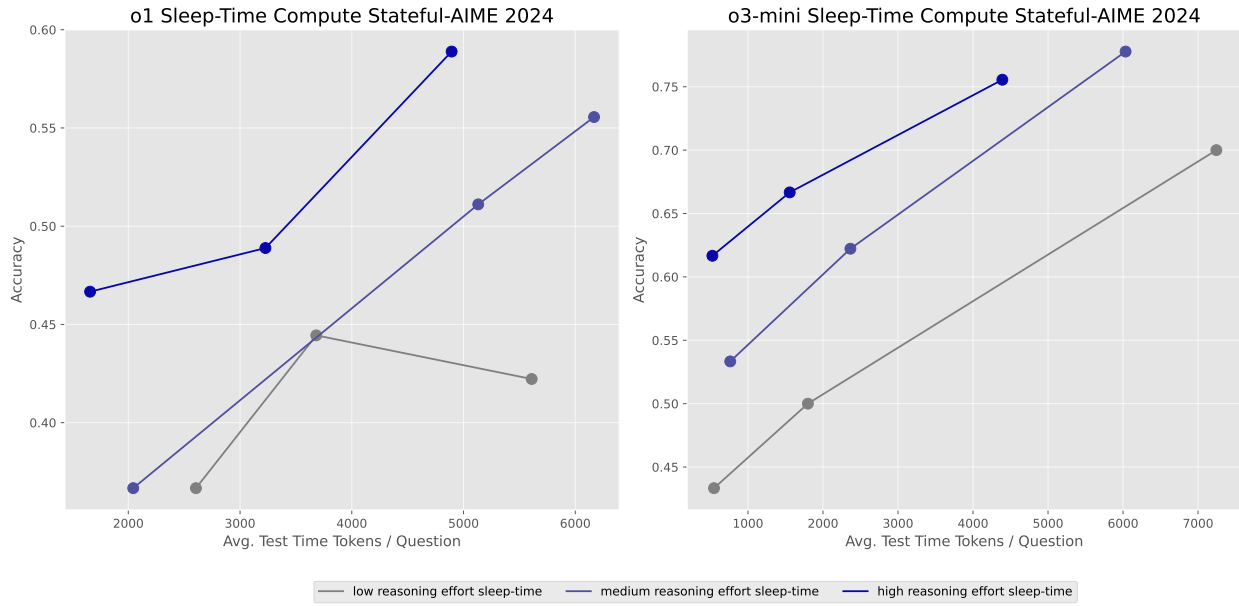


Figure 25: Scaling sleep-time compute for Stateful AIME2024.

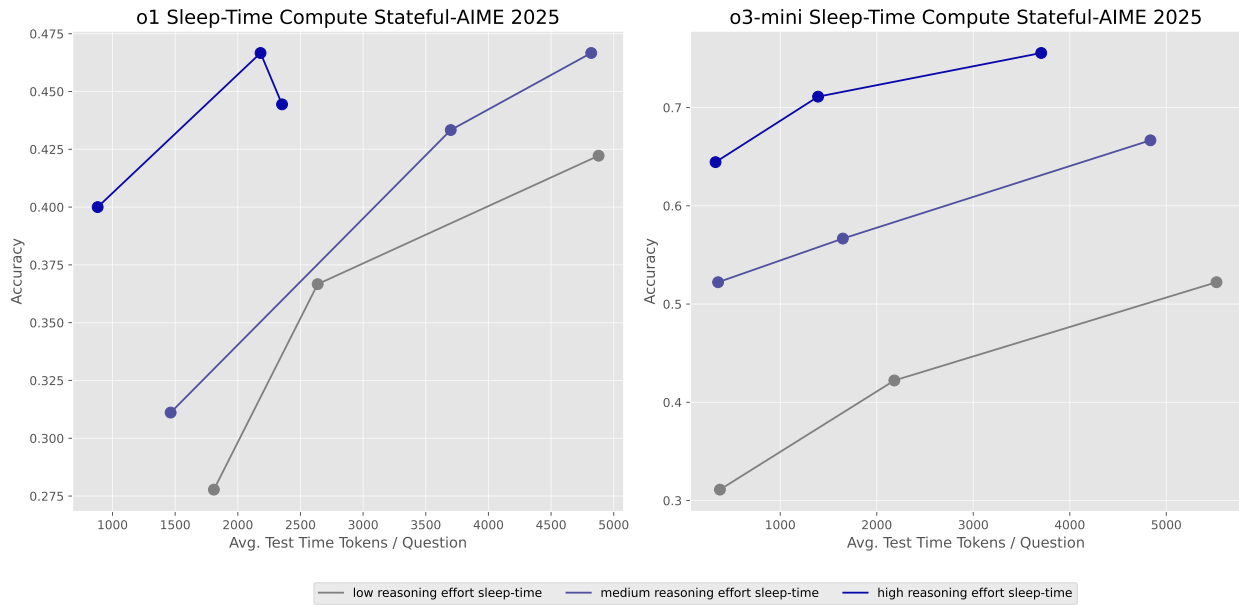


Figure 26: Scaling sleep-time compute on Stateful AIME2025